UNIVERSIDADE DO ESTADO DE MINAS GERAIS – UEMG

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

JOÃO VICTOR COSTA ARRUDA

ROBERTO HENRIQUE DUARTE

SISTEMA PWA PREDITIVO PARA GESTÃO DE FROTAS E VALIDAÇÃO DE EMBARQUE ESTUDANTIL INTERMUNICIPAL

Passos-MG

2026

JOÃO VICTOR COSTA ARRUDA

ROBERTO HENRIQUE DUARTE

SISTEMA PWA PREDITIVO PARA GESTÃO DE FROTAS E VALIDAÇÃO DE EMBARQUE ESTUDANTIL INTERMUNICIPAL

Trabalho de Conclusão de Curso apresentado junto ao curso de Sistemas de Informação, da Universidade do Estado de Minas Gerais - Campus Passos, como requisito parcial à obtenção do título de Bacharel em Sistemas de Informação.

Orientador: Prof. Eduardo Henrique Marques Ferreira.

Passos-MG

2026

JOÃO VICTOR COSTA ARRUDA
ROBERTO HENRIQUE DUARTE

SISTEMA PWA PREDITIVO PARA GESTÃO DE FROTAS E VALIDAÇÃO DE EMBARQUE ESTUDANTIL INTERMUNICIPAL

Trabalho de Conclusão de Curso apresentado ao curso de Sistemas de Informação, da Universidade do Estado de Minas Gerais - Campus Passos, como parte das exigências para obtenção do título de Bacharel em Sistemas de Informação.

Área de concentração: Sistemas de Informação

Banca examinadora:

_____

Orientador: Prof. Eduardo Henrique Marques Ferreira
UEMG/Campus Passos

_____

Membro: Prof.ª
UEMG/Campus Passos

_____

Membro: Prof.
UEMG/Campus Passos

Passos-MG
2026

Dedico este trabalho aos meus pais e familiares, pelo apoio incondicional, pelos valores transmitidos e por sempre acreditarem no poder transformador da educação, e aos meus amigos, pelo incentivo e companheirismo ao longo de toda a minha jornada acadêmica.

João Victor Costa Arruda

Dedico esta conquista à minha família, alicerce fundamental em todas as etapas da minha formação, pela paciência, compreensão e encorajamento diário, e aos amigos que compartilharam desafios e aprendizados durante esta trajetória universitária.

Roberto Henrique Duarte

AGRADECIMENTOS

Agradecemos primeiramente a Deus e aos nossos familiares, pelo suporte incondicional, pelos princípios ensinados e pelo incentivo constante que nos sustentaram ao longo de toda a caminhada universitária. Aos nossos amigos e colegas de graduação, registramos nossa gratidão pela parceria, pela troca de conhecimentos e pelo apoio mútuo nos momentos mais desafiadores da formação.

Ao nosso orientador, Prof. Eduardo Henrique Marques Ferreira, expressamos especial reconhecimento pela condução segura, pela paciência, pela disponibilidade e pelas valiosas orientações técnicas e metodológicas, fundamentais para a concepção, o amadurecimento e a consolidação deste trabalho.

Estendemos nossa gratidão ao corpo docente e aos servidores do Curso de Bacharelado em Sistemas de Informação da Universidade do Estado de Minas Gerais, Campus Passos, pelos ensinamentos compartilhados e pelo compromisso com o ensino público de qualidade. Por fim, aos estudantes e profissionais do transporte universitário intermunicipal pelas percepções compartilhadas acerca da rotina de deslocamento acadêmico, nosso sincero agradecimento.

“A imaginação é mais importante que o conhecimento. O conhecimento é limitado, enquanto a imaginação abraça o mundo inteiro.”

(Albert Einstein, 1929)

RESUMO

O transporte universitário intermunicipal enfrenta gargalos operacionais recorrentes, como a superlotação de veículos e a ociosidade de frotas, decorrentes da dependência de controles analógicos e da ausência de conferência nominal no momento do embarque. Essa ineficiência evidencia a necessidade de um planejamento adequado da mobilidade coletiva que promova a alocação racional de recursos e assegure a prestação de um serviço seguro e de qualidade aos discentes. Para mitigar essas falhas, este trabalho tem como objetivo geral desenvolver o sistema AppBus, uma plataforma computacional integrada para a gestão operacional e o controle de frequência estudantil. O estudo adota metodologia de pesquisa aplicada e exploratória sob as diretrizes da *Design Science Research* (DSR), com foco na concepção e construção de um Produto Mínimo Viável (MVP). A arquitetura desacoplada da solução integra um *frontend* no formato *Progressive Web App* (PWA) em Vue.js 3 com Quasar Framework, um *backend* RESTful em Python com Django REST Framework sob os padrões *Party/Role* e *multi-tenant*, banco de dados relacional PostgreSQL e orquestração de ambiente em contêineres Docker. A aplicação disponibiliza predição dinâmica de demanda na modalidade "Vou/Não vou" com gestão de prioridade de ciclo e fila de espera no retorno, embarque óptico via QR Code, contingência *offline-first* para conferência nominal pelo motorista e mural de avisos assíncronos. Adicionalmente, o sistema implementa governança de privacidade em conformidade com a Lei Geral de Proteção de Dados Pessoais (Lei nº 13.709/2018 — LGPD), garantindo o registro de prova de consentimento e a anonimização irreversível de dados pessoais. A validação funcional do artefato demonstra que a plataforma viabiliza previsibilidade exata de ocupação para o dimensionamento eficiente das frotas públicas e fretadas, reduz falhas operacionais em rotas sem conectividade e fortalece a rastreabilidade e a segurança no translado universitário.

Palavras-chave: Sistemas de Informação. Transporte Universitário. Progressive Web App. Previsão de Demanda. Lei Geral de Proteção de Dados.

ABSTRACT

Intercity university transportation faces recurrent operational bottlenecks, such as vehicle overcrowding and fleet idleness, stemming from reliance on analog controls and the absence of nominal verification during boarding. This inefficiency highlights the need for effective collective mobility planning that promotes rational resource allocation and ensures safe, high-quality service delivery for students. To mitigate these shortcomings, this study aims to develop the AppBus system, an integrated software platform for operational management and student attendance tracking. The study adopts an applied and exploratory research methodology following Design Science Research (DSR) guidelines, focusing on the design and construction of a Minimum Viable Product (MVP). The decoupled architecture of the solution integrates a Progressive Web App (PWA) frontend in Vue.js 3 with Quasar Framework, a RESTful Python backend with Django REST Framework under the Party/Role and multi-tenant patterns, a PostgreSQL relational database, and environment orchestration in Docker containers. The application provides dynamic "Going/Not Going" demand prediction with cycle priority management and a return-trip waiting list, optical QR Code boarding validation, offline-first contingency operation for nominal verification by the driver, and an asynchronous notice board. Additionally, the system implements privacy governance in compliance with the Brazilian General Data Protection Law (Law No. 13,709/2018 — LGPD), ensuring the recording of proof of consent and the irreversible anonymization of personal data. Functional validation of the artifact demonstrates that the platform enables accurate occupancy predictability for the efficient sizing of public and chartered fleets, reduces operational failures along routes without connectivity, and strengthens traceability and safety in university transit.

Keywords: Information Systems. University Transport. Progressive Web App. Demand Prediction. Data Protection.

LISTA DE FIGURAS

Figura 1 – Página inicial da plataforma MeuEscolar 16
Figura 2 – Tela de autenticação do aplicativo VanEscola..... 17
Figura 3 – Tela de cadastro de rota da plataforma VanEscola 17
Figura 4 – Painel gerencial do aplicativo Clickvan..... 18
Figura 5 – Diagrama de Casos de Uso do sistema AppBus 35
Figura 6 – Diagrama Entidade-Relacionamento (DER) do sistema AppBus..... 39
Figura 7 – Arquitetura inicial e orquestração de contêineres do sistema AppBus..... 40
Figura 8 – Tela de autenticação multi-perfil do sistema AppBus 42
Figura 9 – Tela de auto-cadastro discente com validação condicional LGPD..... 43
Figura 10 – Painel administrativo do gestor municipal..... 45
Figura 11 – Painel operacional de bordo do motorista e contingência offline..... 46
Figura 12 – Painel principal do estudante com carteirinha digital e prioridade de ciclo 48
Figura 13 – Leitor óptico de QR Code e painel de autogestão de privacidade LGPD..... 49
Figura 14 – Mural de avisos e comunicados assíncronos com respostas rápidas 50

LISTA DE QUADROS

Quadro 1 – Comparativo de soluções comerciais de gestão de transporte 19
Quadro 2 – Comparativo de trabalhos acadêmicos correlatos e o sistema AppBus 20
Quadro 3 – Requisitos Funcionais (RF) do sistema AppBus 30
Quadro 4 – Requisitos Não Funcionais (RNF) do sistema AppBus 32
Quadro 5 – Regras de Negócio (RN) do sistema AppBus 34
Quadro 6 – Dicionário de entidades do Modelo Entidade-Relacionamento (ERM) do AppBus 37

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	*Application Programming Interface* (Interface de Programação de Aplicações)
BI	*Business Intelligence* (Inteligência de Negócios)
CNH	Carteira Nacional de Habilitação
CORS	*Cross-Origin Resource Sharing* (Compartilhamento de Recursos de Origem Cruzada)
CPF	Cadastro de Pessoas Físicas
CSS	*Cascading Style Sheets* (Folhas de Estilo em Cascata)
DER	Diagrama Entidade-Relacionamento
DRF	*Django REST Framework*
DSR	*Design Science Research*
ECA	Estatuto da Criança e do Adolescente
ERM	*Entity-Relationship Model* (Modelo Entidade-Relacionamento)
FAB	*Floating Action Button* (Botão de Ação Flutuante)
GPS	*Global Positioning System* (Sistema de Posicionamento Global)
HTML	*HyperText Markup Language* (Linguagem de Marcação de Hipertexto)
HTTP	*Hypertext Transfer Protocol* (Protocolo de Transferência de Hipertexto)
IDE	*Integrated Development Environment* (Ambiente de Desenvolvimento Integrado)
IP	*Internet Protocol* (Protocolo de Internet)
ISO	*International Organization for Standardization* (Organização Internacional de Normalização)
JSON	*JavaScript Object Notation* (Notação de Objetos JavaScript)
JWT	*JSON Web Token*
KPI	*Key Performance Indicator* (Indicador-Chave de Desempenho)
LGPD	Lei Geral de Proteção de Dados Pessoais
MVCC	*Multi-Version Concurrency Control* (Controle de Concorrência Multiversão)
MVP	*Minimum Viable Product* (Produto Mínimo Viável)
MVVM	*Model-View-ViewModel*
ORM	*Object-Relational Mapping* (Mapeamento Objeto-Relacional)
PBKDF2	*Password-Based Key Derivation Function 2* (Função de Derivação de Chave Baseada em Senha 2)
PDF	*Portable Document Format* (Formato Portátil de Documento)
PNG	*Portable Network Graphics* (Gráficos de Rede Portáteis)

PWA	*Progressive Web App* (Aplicativo Web Progressivo)
QR Code	*Quick Response Code* (Código de Resposta Rápida)
RA	Registro Acadêmico
RBAC	*Role-Based Access Control* (Controle de Acesso Baseado em Papéis)
REST	*Representational State Transfer* (Transferência de Estado Representacional)
RF	Requisito Funcional
RN	Regra de Negócio
RNF	Requisito Não Funcional
SGBDOR	Sistema Gerenciador de Banco de Dados Objeto-Relacional
SHA	*Secure Hash Algorithm*
SPA	*Single Page Application* (Aplicação de Página Única)
SQL	*Structured Query Language* (Linguagem de Consulta Estruturada)
SUS	*System Usability Scale* (Escala de Usabilidade do Sistema)
TI	Tecnologia da Informação
UF	Unidade Federativa
UI	*User Interface* (Interface do Usuário)
UML	*Unified Modeling Language* (Linguagem de Modelagem Unificada)
URL	*Uniform Resource Locator* (Localizador Uniforme de Recursos)
UUID	*Universally Unique Identifier* (Identificador Único Universal)
WSGI	*Web Server Gateway Interface*

SUMÁRIO

1. INTRODUÇÃO **12**
1.1. Justificativa 13
1.2. Objetivos 14
1.2.1. Objetivo geral 14
1.2.2. Objetivos específicos 14
1.3. Metodologia 15
1.4. Organização do trabalho 15
2. REFERENCIAL TEÓRICO **16**
2.1. Mobilidade urbana e transporte público no Brasil 16
2.2. Transporte universitário e previsão de demanda 16
2.3. Sistemas de controle de embarque de passageiros 16
2.3.1. MeuEscolar 16
2.3.2. VanEscola 17
2.3.3. Clickvan 18
2.4. Arquitetura de software e tecnologias web 21
2.5. Metodologia de pesquisa: Design Science Research (DSR) 22
2.6. Lei Geral de Proteção de Dados (LGPD) no contexto de sistemas web 23
3. MATERIAIS E MÉTODOS **25**
3.1. Python 27
3.2. Django e Django REST Framework 27
3.3. API RESTful 28
3.4. PostgreSQL 28
3.5. Vue.js e Quasar Framework 28
3.6. Progressive Web App (PWA) 29
3.7. Docker 29
3.8. Google Antigravity IDE 29
4. APRESENTAÇÃO DO PROJETO **30**
4.1. Requisitos de software 30
4.1.1. Requisitos funcionais 30
4.1.2. Requisitos não funcionais 32
4.1.3. Regras de negócio 34
4.2. Diagrama de casos de uso 35
4.3. Modelagem do banco de dados 36
4.4. Arquitetura inicial do sistema 40
4.5. Protótipos de telas e interfaces do sistema 41
4.5.1. Telas de autenticação, auto-cadastro e consentimento LGPD 41

4.5.2. Painel administrativo do gestor 44
4.5.3. Painel operacional de bordo do motorista 46
4.5.4. Painel do estudante, predição, leitura óptica de QR Code e privacidade LGPD 47
4.5.5. Mural de avisos e comunicados assíncronos 50
5. CONSIDERAÇÕES FINAIS **51**
REFERÊNCIAS **53**

1. INTRODUÇÃO

O uso das Tecnologias da Informação (TI) tornou-se elemento indispensável nos processos produtivos e na gestão organizacional, impulsionando a desburocratização de rotinas e a otimização de serviços. Dentre os serviços públicos e comunitários que demandam aprimoramento tecnológico contínuo, a mobilidade urbana e o transporte coletivo destacam-se como vetores estruturantes para o desenvolvimento social e para a garantia do acesso à educação e aos serviços urbanos, conforme discutido por Carvalho et al. (2023) e Kneib (2022). Nesse contexto, o planejamento adequado da mobilidade urbana impõe desafios constantes na busca pela eficiência na alocação de recursos do transporte coletivo e pela prestação de um serviço público de qualidade à população (VASCONCELLOS, 2014).

No contexto específico das instituições de ensino superior, o transporte universitário intermunicipal assume papel decisivo na rotina dos discentes. Contudo, a gestão desse transporte frequentemente enfrenta gargalos operacionais relevantes, pautados pela imprevisibilidade da demanda e pela ausência de mecanismos automatizados de controle de presença. Conforme apontam Hamad e Obaid (2022), os viajantes universitários possuem padrões de deslocamento únicos, exigindo modelos específicos de previsão de demanda para evitar distorções operacionais. A falta de previsão exata da quantidade de passageiros gera episódios recorrentes de superlotação, enquanto a alocação superestimada de veículos resulta em ociosidade da frota e despesas orçamentárias desnecessárias para prefeituras e associações, reforçando a necessidade de modelos preditivos na gestão de transportes (BANERJEE; MORTON; AKARTUNALI, 2020).

Muitas organizações e cooperativas de transporte ainda dependem de métodos arcaicos de controle, como listas físicas em papel ou grupos informais em aplicativos de mensagens instantâneas. Tais métodos mostram-se ineficazes pela vulnerabilidade a falhas humanas, rasuras e ausência de consolidação de dados em tempo real. Soma-se a esse cenário a exigência legal de adequação às diretrizes da Lei Geral de Proteção de Dados Pessoais — Lei nº 13.709/2018 (BRASIL, 2018), que impõe padrões rigorosos para a coleta, o consentimento e o armazenamento de dados pessoais dos estudantes (DONEDA, 2020).

Para mitigar essas falhas operacionais e de conformidade legal, surge a proposta do AppBus, um sistema computacional concebido no formato *Progressive Web App* (PWA) integrado a uma API RESTful baseada nos princípios arquiteturais definidos por Fielding (2000). A solução foi projetada para unificar a predição contínua e dinâmica de embarque

discente ("Vou/Não vou") com gestão de prioridade de ciclo e fila de espera no retorno, a validação de presença via leitura óptica de QR Code, o controle de contingência para motoristas, a comunicação operacional assíncrona via mural de avisos e um módulo de governança e privacidade voltado ao cumprimento integral das exigências da LGPD (BRASIL, 2018).

1.1. Justificativa

A automatização e a sistematização da gestão do transporte universitário intermunicipal justificam-se sob os âmbitos social, econômico, tecnológico e legal.

Sob a perspectiva social e acadêmica, o AppBus assegura o direito fundamental do estudante à mobilidade e ao acesso à universidade, eliminando o risco de o passageiro ser deixado para trás por falhas de contagem manual do motorista. Sob o aspecto econômico, a disponibilização de um painel preditivo permite o dimensionamento preciso da frota por parte de prefeituras e empresas concessionárias, reduzindo custos operacionais e otimizando o emprego de recursos públicos e privados (KNEIB, 2022; HAMAD; OBAID, 2022).

Do ponto de vista tecnológico, a escolha da arquitetura PWA (*Progressive Web App*) desenvolvida em Vue.js 3 com Quasar Framework justifica-se pela pluralidade socioeconômica do público discente. Exigir o *download* de aplicativos nativos pesados (de 50 MB a 100 MB) em *stores* de aplicativos imporia uma barreira de acesso a estudantes de baixa renda com *smartphones* de entrada. Como demonstrado por Biørn-Hansen, Majchrzak e Grønli (2017) e Tandel e Jamadar (2018), os PWAs possuem um tamanho de instalação drasticamente reduzido e rodam diretamente no navegador. Além disso, a instalação de *Service Workers* permite a execução de rotinas em *background* e oferece suporte a funcionalidades *offline-first*, essencial para garantir a operabilidade do motorista em rodovias com sinal móvel instável (BIØRN-HANSEN; MAJCHRZAK; GRØNLI, 2017). No *backend*, a utilização da linguagem Python com o *framework* Django REST Framework, aliada à arquitetura de APIs RESTful (FIELDING, 2000), ao banco de dados PostgreSQL e à orquestração via Docker, garante alto desempenho, isolamento do ambiente e reuso de componentes de software (SOMMERVILLE, 2018).

No âmbito legal, o projeto justifica-se pela urgente necessidade de conformidade com a LGPD (BRASIL, 2018). O AppBus implementa o princípio da minimização de dados (Art. 6º, III), limitando a coleta ao estritamente necessário para a prestação do serviço; registra provas digitais de consentimento com endereço IP e *user-agent* em conformidade com o Art. 8º; prevê tratamento condicional para menores de idade nos termos do Art. 14; e adota uma estratégia de

anonimização irreversível fundamentada nos Artigos 12 e 18 da lei, permitindo que o aluno revogue seus dados pessoais sem comprometer o histórico estatístico e auditável das viagens efetuadas pelo município (DONEDA, 2020).

1.2. Objetivos

1.2.1. Objetivo geral

Desenvolver um sistema web progressivo (PWA) integrado a uma API RESTful para a gestão preditiva de frotas e controle de embarque no transporte universitário intermunicipal, garantindo a eficiência operacional e a conformidade com a LGPD.

1.2.2. Objetivos específicos

- Mapear os requisitos funcionais e não funcionais inerentes à rotina do transporte universitário e às exigências de privacidade da LGPD;
- Projetar a arquitetura de software desacoplada (Vue.js 3 e Quasar Framework no *frontend*, Python com Django REST Framework no *backend*, PostgreSQL e Docker) baseada nos padrões *Party/Role* e *multi-tenant*;
- Implementar o módulo de predição de demanda contínua e dinâmica ("Vou/Não vou") com controle de prioridade de ciclo para o trajeto de volta e fila de espera automática para vagas remanescentes, a validação de embarque via QR Code, a contingência de *check-in* manual do motorista e a disponibilização de um mural de avisos e comunicados operacionais por município e rota;
- Estruturar o Módulo de Governança e Privacidade para o registro de provas de consentimento, controle de menores de idade e fluxo de anonimização;
- Avaliar a usabilidade e o desempenho do Produto Mínimo Viável (MVP) com apoio da escala *System Usability Scale* (SUS) e dos parâmetros de qualidade da norma ISO 9241-11:2018.

1.3. Metodologia

A condução deste trabalho caracteriza-se como uma pesquisa aplicada e exploratória. Conforme apontado por Gil (2019) e Piovesan e Temporini (1995), a pesquisa exploratória e qualitativa é fundamental para a delimitação de problemas complexos, permitindo mapear variáveis e fundamentar a tomada de decisões em cenários organizacionais.

A abordagem metodológica central apoia-se nos princípios da *Design Science Research* (DSR), proposta por Hevner et al. (2004). A DSR é orientada à solução de problemas práticos por meio da concepção, desenvolvimento e avaliação iterativa de artefatos computacionais relevantes para a área de Sistemas de Informação. O ciclo de desenvolvimento compreende o levantamento de requisitos de software (SOMMERVILLE, 2018), a modelagem relacional do banco de dados, a implementação da API e das interfaces PWA do AppBus, finalizando com a validação do MVP mediante testes de usabilidade baseados na escala SUS (BROOKE, 1996).

1.4. Organização do trabalho

O presente Trabalho de Conclusão de Curso encontra-se estruturado em cinco capítulos, distribuídos da seguinte forma:

- **Capítulo 1 (Introdução):** Apresenta a contextualização do tema, a problematização, a justificativa da proposta AppBus, os objetivos geral e específicos, a metodologia de pesquisa e a organização do documento.
- **Capítulo 2 (Referencial Teórico):** Reúne a fundamentação conceitual sobre mobilidade urbana (CARVALHO et al., 2023; KNEIB, 2022), transporte público universitário e previsão de demanda (HAMAD; OBAID, 2022; BANERJEE; MORTON; AKARTUNALI, 2020), arquitetura de APIs RESTful (FIELDING, 2000), *Progressive Web Apps* (BIØRN-HANSEN; MAJCHRZAK; GRØNLI, 2017), a metodologia *Design Science Research* (HEVNER et al., 2004) e as normativas da Lei Geral de Proteção de Dados (BRASIL, 2018; DONEDA, 2020).
- **Capítulo 3 (Materiais e Métodos):** Detalha os instrumentos, linguagens, *frameworks* (Vue.js 3, Quasar Framework, Django REST Framework, PostgreSQL, Docker) e os procedimentos práticos adotados na condução do projeto.
- **Capítulo 4 (Apresentação do Projeto e Resultados):** Exibe a especificação de requisitos de software, diagramas UML (casos de uso, classes/DER), as telas e protótipos do sistema AppBus, bem como os fluxos de predição com prioridade de ciclo e fila de espera, validação por QR Code, mural de avisos operacionais, o Módulo de Privacidade e a avaliação dos testes do MVP.
- **Capítulo 5 (Considerações Finais):** Apresenta as conclusões obtidas com a execução do trabalho, sintetiza as contribuições do artefato e sugere direcionamentos para trabalhos futuros.

2. REFERENCIAL TEÓRICO

2.1. Mobilidade urbana e transporte público no Brasil

A mobilidade urbana constitui um elemento estruturante para o desenvolvimento socioeconômico das cidades contemporâneas, atuando como vetor indispensável para a garantia do direito de circulação e do acesso a serviços fundamentais, tais como saúde, trabalho e educação (CARVALHO et al., 2023; KNEIB, 2022). O panorama do transporte no Brasil reflete desigualdades socioespaciais históricas, nas quais o transporte público coletivo enfrenta desafios operacionais e institucionais que impactam diretamente a qualidade de vida da população urbana e intermunicipal (CARVALHO et al., 2023).

Conforme analisa Kneib (2022), os gargalos do transporte público coletivo no cenário nacional podem ser compreendidos a partir de quatro eixos basilares:

1. **Financiamento:** Necessidade de estruturação de modelos sustentáveis de custeio e de gestão de subsídios extratarifários;
2. **Infraestrutura:** Disponibilidade e manutenção de vias prioritárias, terminais de integração e equipamentos de apoio à circulação;
3. **Uso e Ocupação do Solo:** Articulação entre o planejamento urbano e os eixos de adensamento populacional;
4. **Governança e Gestão Pública:** Capacidade técnica e administrativa do setor público para o planejamento, a regulação, o monitoramento e a fiscalização dos serviços prestados.

A fragilidade na integração desses eixos resulta na perda de eficiência do sistema de transporte, gerando custos operacionais elevados e assimetria no acesso aos serviços urbanos (KNEIB, 2022). Diante desse cenário, a incorporação de sistemas de informação e tecnologias de comunicação surge como instrumento de apoio à gestão pública, permitindo o levantamento de dados de circulação para subsidiar o planejamento logístico de frotas (CARVALHO et al., 2023; KNEIB, 2022).

2.2. Transporte universitário e previsão de demanda

No contexto das instituições de ensino superior, o transporte universitário intermunicipal desempenha papel relevante na garantia da frequência discente. O perfil socioeconômico dos estudantes de graduação evidencia a dependência do transporte público e

comunitário para a continuidade dos estudos universitários (FRANCO; CUNHA, 2014). Todavia, conforme destacam Hamad e Obaid (2022), os passageiros universitários possuem padrões de deslocamento singulares (*activity-based/tour-based*), caracterizados por alta variabilidade em função das grades horárias, dias letivos, períodos de exames e compromissos acadêmicos assíncronos. Essa dinâmica distingue o transporte universitário dos fluxos pendulares tradicionais do transporte coletivo urbano (HAMAD; OBAID, 2022).

A ausência de acompanhamento da demanda real resulta em ineficiências operacionais: a subestimativa da ocupação gera episódios de superlotação e exclusão de passageiros nos pontos de embarque, enquanto a alocação superestimada de veículos acarreta ociosidade da frota, custos operacionais desnecessários e emissões evitáveis de poluentes (BANERJEE; MORTON; AKARTUNALI, 2020). De acordo com a revisão sistemática de Banerjee, Morton e Akartunali (2020), a estimativa e a previsão contínua de passageiros no transporte agendado são aspectos críticos para o dimensionamento de capacidade, gerenciamento do inventário de assentos e planejamento orçamentário.

Para fins de precisão científica no âmbito deste trabalho, diferenciam-se três conceitos correlatos:

- **Intenção de Viagem:** Declaração voluntária do passageiro referente à sua decisão de utilizar o serviço em determinado dia e horário (ex.: confirmação "Vou/Não vou");
- **Estimativa de Passageiros:** Contagem quantitativa acumulada resultante da consolidação das intenções registradas em determinado período;
- **Previsão de Demanda:** Aplicação de modelos estatísticos, econométricos ou estocásticos para projetar cenários futuros com base em séries históricas e variáveis explicativas (BANERJEE; MORTON; AKARTUNALI, 2020).

O AppBus atua na camada de captura contínua da intenção de viagem do discente ("Vou/Não vou") por meio de atualizações parciais (*PATCH*) na tabela transacional *Previsao_Viagem* (entidade *PrevisaoViagem*, restrita por unicidade composta nos atributos *aluno* e *data*, com indicadores booleanos independentes *vai_ida* e *vai_volta*), disponibilizando aos gestores e motoristas uma estimativa quantitativa da lotação em tempo real via agregação SQL (*COUNT*), aliada ao controle algorítmico de prioridade de ciclo (garantia de retorno aos passageiros da ida e fila de espera cronológica para vagas remanescentes), sem a necessidade de processamento preditivo baseado em inteligência artificial na etapa de MVP.

2.3. Sistemas de controle de embarque de passageiros

Para a escolha dos principais sistemas comerciais de controle de embarque e gestão de transporte de passageiros, foi realizada uma pesquisa por aplicações de mercado voltadas ao transporte escolar e fretamento universitário. Foram selecionadas plataformas consolidadas que oferecem funcionalidades de rastreamento veicular, chamadas digitais e aplicativos móveis.

2.3.1. MeuEscolar

O aplicativo MeuEscolar é uma ferramenta voltada para motoristas e empresas de transporte escolar (MEUESCOLAR, 2026). A plataforma centraliza a operação, oferecendo o controle de alunos, gestão financeira (mensalidades e despesas), rotas inteligentes com rastreamento via GPS em tempo real e chamadas digitais de presença. O sistema também inclui um aplicativo exclusivo para os pais ou responsáveis, permitindo o acompanhamento do trajeto e o recebimento de notificações, conforme ilustrado na Figura 1.

Figura 1 – Página inicial da plataforma MeuEscolar

Inserir imagem da Figura 1 – Página inicial da plataforma MeuEscolar

Fonte: MeuEscolar (2026).

Apesar de ser uma ferramenta robusta, a aplicação possui foco primordial no modelo de prestação privada de serviços, dependendo de rotas fixas e sem evidenciar um sistema dinâmico de intenção de viagem ("Vou/Não vou") adaptável à rotina universitária.

2.3.2. VanEscola

O aplicativo VanEscola apresenta-se como uma solução focada na gestão integral de vans escolares, integrando rotinas administrativas com o acompanhamento operacional (VANESCOLA, 2026). O sistema dispõe de recursos de rastreamento veicular em tempo real (com suporte ao Waze), automação de contratos e notificações instantâneas de embarque e desembarque, contando com tela dedicada de autenticação no dispositivo móvel, conforme ilustrado na Figura 2.

Figura 2 – Tela de autenticação do aplicativo VanEscola

Inserir imagem da Figura 2 – Tela de autenticação do aplicativo VanEscola

Fonte: VanEscola (2026).

Além do aplicativo móvel, o sistema conta com um painel administrativo acessível via navegador *web* para o gerenciamento logístico (VANESCOLA, 2026). Por meio do módulo de cadastro de rotas, o gestor pode definir o nome do trajeto, registrar o horário de início, alocar os veículos e parametrizar os dias da semana de operação, como dias úteis de segunda a sexta-feira, conforme ilustrado na Figura 3.

Figura 3 – Tela de cadastro de rota da plataforma VanEscola

Inserir imagem da Figura 3 – Tela de cadastro de rota da plataforma VanEscola

Fonte: VanEscola (2026).

2.3.3. Clickvan

A plataforma Clickvan é um *software* de gestão que oferece um ecossistema integrado para a administração do transporte (CLICKVAN, 2026). A ferramenta permite centralizar a operação logística e financeira, disponibilizando um aplicativo móvel voltado para a equipe operacional. O painel de controle do aplicativo atua como um *dashboard* gerencial móvel, permitindo ao usuário acompanhar os principais indicadores do mês de referência, conforme ilustrado na Figura 4.

Figura 4 – Painel gerencial do aplicativo Clickvan

Inserir imagem da Figura 4 – Painel gerencial do aplicativo Clickvan

Fonte: Clickvan (2026).

A literatura acadêmica e o mercado de *software* apresentam diversas aplicações voltadas ao controle de acesso, fiscalização e acompanhamento do transporte de passageiros. No âmbito dos sistemas comerciais para transporte escolar e fretamento (MeuEscolar, VanEscola e Clickvan), as plataformas disponibilizam funcionalidades essenciais de rastreamento veicular, controle de rotas, chamadas digitais de presença e gestão financeira (CLICKVAN, 2026; MEUESCOLAR, 2026; VANESCOLA, 2026). Na documentação e nas interfaces consultadas

dessas soluções comerciais, observa-se foco primordial no modelo de prestação privada de serviços, sem evidência de mecanismos integrados de governança da LGPD voltados especificamente à administração pública municipal ou de módulos automáticos de confirmação de presença dinâmica sem amarragem de horários rígidos, conforme sintetizado no Quadro 1.

Quadro 1 – Comparativo de soluções comerciais de gestão de transporte

Critério de Análise	Clickvan	VanEscola	MeuEscolar
Plataforma Principal	Sistema *Web* / *App*	Sistema *Web* / *App*	Sistema *Web* / *App*
Público-Alvo	Transporte Escolar / Frotas	Transporte Escolar	Transporte Escolar
Coleta de Intenção ("Vou/Não vou")	Não identificada	Aviso de Ausência	Chamada do Motorista
Validação de Embarque	Chamada no *App*	Notificação de Embarque	Chamada Manual
Operação Offline-First (Motorista)	Não identificada	Não identificada	Não identificada
Arquitetura Multi-tenant	Comercial (Privado)	Comercial (Privado)	Comercial (Privado)
Módulo de Governança LGPD	Aceite de Termos	Aceite de Termos	Não identificada

Fonte: Elaborado pelos autores (2026).

No campo acadêmico e institucional, identificam-se estudos e protótipos com escopos correlatos:

- **Solução Unicamp (SOUSA; BARBOSA; HINZ, 2022):** Sistema *web* integrado a um aplicativo para leitura de QR Code em dispositivos de fiscais, voltado à validação e fiscalização de usuários cadastrados no transporte fretado universitário;
- **Projeto de Pinto (2017):** Protótipo de aplicativo móvel para o sistema operacional Android focado na comunicação entre motoristas e responsáveis do transporte escolar, permitindo a indicação manual de ausências discentes;
- **Sistema IFFar Uruguaiana (DIAS, 2023):** Sistema *web* desenvolvido em PHP/Bootstrap integrado a leitor de QR Code para monitoramento do fluxo de entrada e saída de estudantes nas portarias do *campus*;
- **AcadBus (MOREIRA; SPAGNOL; BARBOSA, 2024):** Aplicativo direcionado ao controle de acesso e acompanhamento de acadêmicos no transporte universitário.

O AppBus diferencia-se dos trabalhos analisados pela integração de cinco elementos funcionais em uma única arquitetura:

1. Interface *Progressive Web App* (PWA) de baixo consumo de *hardware*;

2. Declaração dinâmica de intenção de viagem ("Vou/Não vou") atualizada continuamente;

3. Validação dupla de embarque (leitura óptica de QR Code pelo aluno ou *check-in* manual de contingência pelo motorista);

4. Arquitetura *backend multi-tenant* para segregação de dados entre diferentes clientes/municípios;

5. Módulo de governança de dados pessoais integrado conforme a LGPD.

O Quadro 2 apresenta a síntese comparativa entre os trabalhos acadêmicos correlatos analisados e o sistema AppBus proposto.

Quadro 2 – Comparativo de trabalhos acadêmicos correlatos e o sistema AppBus

Critério de Análise	Solução Unicamp (SOUSA; BARBOSA; HINZ, 2022)	Projeto de Pinto (2017)	Sistema IFFar (DIAS, 2023)	AcadBus (MOREIRA; SPAGNOL; BARBOSA, 2024)	AppBus (Proposto)
Plataforma Principal	Sistema *Web* / *App*	*App* Nativo (Android)	Sistema *Web*	Aplicativo Móvel	PWA (Vue.js) + API REST
Público-Alvo	Fretado Universitário	Motorista Escolar	Portaria do *Campus*	Transporte Universitário	Transporte Universitário Intermunicipal
Coleta de Intenção ("Vou/Não vou")	Solicitação de Uso	Indicação de Ausência	Ausente	Confirmação de Presença	Dinâmica Contínua (Ida/Volta independente)
Validação de Embarque	Leitura de QR Code	Lista no *App*	Leitura de QR Code	Controle de Acesso no *App*	Dupla (QR Code pelo Aluno + Manual pelo Motorista)
Operação Offline-First (Motorista)	Não identificada	Ausente	Ausente	Não identificada	Sim (*Service Worker* + *localStorage* / *Cache API*)
Arquitetura Multi-tenant	Instituição Única	Instituição Única	Instituição Única	Instituição Única	Sim (Segregação por *cliente_id*)

Critério de Análise	Solução Unicamp (SOUSA; BARBOSA; HINZ, 2022)	Projeto de Pinto (2017)	Sistema IFFar (DIAS, 2023)	AcadBus (MOREIRA; SPAGNOL; BARBOSA, 2024)	AppBus (Proposto)
Módulo de Governança LGPD	Parcial (Aprovação)	Ausente	Ausente	Não identificada	Completo (Consentimento IP/UA, Menores e Anonimização)

Fonte: Elaborado pelos autores (2026).

2.4. Arquitetura de software e tecnologias web

O projeto de um sistema de informação exige a seleção de um padrão arquitetural adequado às restrições do ambiente operacional e ao perfil dos usuários (SOMMERVILLE, 2018). O AppBus adota uma arquitetura desacoplada, na qual a interface de usuário (*frontend*) e a camada de regras de negócio/persistência (*backend*) comunicam-se por meio de uma API baseada no estilo arquitetural *Representational State Transfer* (REST) (FIELDING, 2000).

De acordo com Fielding (2000), as restrições do estilo REST — tais como separação cliente-servidor, comunicação sem estado (*stateless*), interface uniforme e *cache* — promovem a escalabilidade do sistema e a independência na evolução dos componentes. Na camada de *backend*, adotou-se a linguagem Python com o *framework* Django REST Framework (DRF), integrada ao banco de dados relacional PostgreSQL. A estrutura transacional foi modelada sob o padrão *Party/Role* — desacoplando as credenciais de autenticação (*UsuarioAuth*) da entidade central de identidade civil (*Pessoa*) e estendendo papéis específicos para *Aluno*, *Motorista* e *Administrador* — com suporte nativo a *Multi-tenancy*. A segregação lógica de dados entre diferentes municípios/clientes é executada na camada de *software*, mediante a aplicação compulsória de filtros pela chave *cliente_id* nas classes base de *ViewSets* (*TenantBaseViewSet*) e controladores do Django. Adicionalmente, a modelagem incorpora o atributo identificador único *qr_code_uuid* (UUIDv4) na entidade *Onibus* para validação óptica de embarque e estrutura a auditoria de privacidade na entidade *Registro_Consentimento* por meio das colunas *consentimento_concedido*, *ip_address*, *user_agent*, *responsavel_legal_cpf* e *data_consentimento*.

Na camada de *frontend*, a aplicação foi desenvolvida em Vue.js no formato *Progressive Web App* (PWA) (BIØRN-HANSEN; MAJCHRZAK; GRØNLI, 2017; TANDEL; JAMADAR, 2018). Conforme analisam Biørn-Hansen, Majchrzak e Grønli (2017), os PWAs utilizam padrões *web* para oferecer experiência próxima às aplicações nativas, permitindo a

instalação na tela inicial sem intermediação obrigatória de lojas digitais de aplicativos (*app stores*).

A adoção do formato PWA fundamenta-se na heterogeneidade dos dispositivos móveis utilizados pelos discentes, reduzindo o tamanho de instalação a alguns *kilobytes* quando comparado ao *download* de aplicações nativas completas (BIØRN-HANSEN; MAJCHRZAK; GRØNLI, 2017; TANDEL; JAMADAR, 2018). Destaca-se que a presença da API de *Service Worker* isoladamente não assegura a operabilidade sem conexão à rede; a capacidade *offline-first* do painel do motorista é obtida pela combinação do *Service Worker* (atuando como *proxy* de rede via Google Workbox) com o modelo arquitetural *App Shell*, o *cache* de ativos estáticos via *Cache API*/*IndexedDB* e a persistência da fila transacional de embarques realizados sem conectividade em *localStorage* (*appbus_offline_checkins*) (BIØRN-HANSEN; MAJCHRZAK; GRØNLI, 2017; TANDEL; JAMADAR, 2018).

A infraestrutura do AppBus é implantada utilizando contêineres Docker, o que assegura o isolamento de dependências, a padronização dos ambientes de desenvolvimento e produção e a reprodutibilidade da implantação (SOMMERVILLE, 2018).

2.5. Metodologia de pesquisa: Design Science Research (DSR)

Quanto ao seu delineamento geral, a presente pesquisa classifica-se como aplicada quanto à natureza e exploratória quanto aos objetivos (GIL, 2019). Segundo Gil (2019), a pesquisa aplicada busca produzir conhecimentos voltados à solução de problemas práticos específicos inseridos em realidades concretas, enquanto a abordagem exploratória possibilita a familiarização com o fenômeno investigado, o aprimoramento de ideias e a delimitação de variáveis em cenários organizacionais complexos.

A condução do desenvolvimento do artefato computacional apoia-se na metodologia *Design Science Research* (DSR), formalizada por Hevner et al. (2004). A DSR constitui um paradigma de pesquisa consolidado na área de Sistemas de Informação que visa solucionar problemas organizacionais relevantes por meio da concepção, construção e avaliação rigorosa de artefatos tecnológicos inovadores ou adaptados a contextos reais (HEVNER et al., 2004).

Conforme estabelecem Hevner et al. (2004), o ciclo metodológico da DSR articula o ambiente de aplicação à base de conhecimento científico a partir de três diretrizes estruturantes:

1. **Relevância do Problema e Concepção do Artefato:** Conexão direta com o ambiente de aplicação para resolver um problema prático relevante — a imprevisibilidade de

ocupação e a fragilidade do controle de embarque no transporte universitário intermunicipal — mediante a construção de um artefato tecnológico viável, materializado neste estudo pelo Produto Mínimo Viável (MVP) do sistema AppBus, composto pela API RESTful e pelas interfaces PWA;

2. **Rigor Científico e Construção Iterativa:** Mobilização contínua da base de conhecimento teórico e técnico consolidado — abrangendo os fundamentos de Engenharia de Software, o estilo arquitetural REST, as aplicações *Progressive Web App* (PWA) e as normativas de governança da LGPD — para orientar as decisões de modelagem e refinar iterativamente os módulos de *software*;
3. **Avaliação Rigorosa do Artefato:** Verificação sistemática da utilidade, da qualidade técnica e da eficácia operacional da solução desenvolvida por meio de testes funcionais, estruturais e métricas padronizadas de validação empírica no contexto de uso pretendido (HEVNER et al., 2004).

De forma complementar à verificação técnica da arquitetura, a avaliação empírica de qualidade da interface do artefato apoia-se em dois referenciais normativos e métricos de usabilidade:

- **Norma ISO 9241-11:2018:** Estabelece a conceituação formal de usabilidade como a medida na qual um sistema, produto ou serviço pode ser utilizado por usuários específicos para alcançar objetivos específicos com eficácia, eficiência e satisfação em um contexto de uso determinado (ISO, 2018);
- **Escala System Usability Scale — SUS:** Desenvolvido por Brooke (1996), o instrumento *System Usability Scale* (SUS) consiste em um questionário padronizado composto por dez afirmações graduadas em escala Likert de cinco pontos, amplamente adotado na engenharia de *software* para mensurar de modo confiável a percepção subjetiva global de usabilidade pelos usuários finais, contemplando neste trabalho os perfis de estudantes e motoristas (BROOKE, 1996).

2.6. Lei Geral de Proteção de Dados (LGPD) no contexto de sistemas web

A centralização do tratamento de dados em plataformas digitais exige a incorporação de salvaguardas técnicas e jurídicas voltadas à proteção da privacidade e à autodeterminação informacional da pessoa natural (DONEDA, 2020). No ordenamento jurídico brasileiro, essa matéria é disciplinada pela Lei Geral de Proteção de Dados Pessoais — Lei nº 13.709/2018

(LGPD), que regulamenta as hipóteses de tratamento, os princípios de segurança e os direitos dos titulares perante controladores públicos e privados (BRASIL, 2018; DONEDA, 2020).

O sistema AppBus adota o paradigma de privacidade desde a concepção (*Privacy by Design*), estruturando um Módulo de Governança e Privacidade diretamente no modelo entidade-relacionamento (ERM) e nas regras de negócio da aplicação para atender às disposições da Lei nº 13.709/2018 por meio de quatro mecanismos técnicos implementados (BRASIL, 2018):

1. **Criptografia de Credenciais e Princípio da Necessidade (Art. 6º, III e VII; Art. 46):** A legislação determina a limitação do tratamento ao mínimo necessário para a realização de suas finalidades, bem como a adoção de medidas técnicas e administrativas aptas a proteger os dados pessoais contra acessos não autorizados (BRASIL, 2018). Em observância à minimização cadastral, a entidade *Pessoa* restringe a coleta aos atributos estritamente indispensáveis à operação do transporte (*nome*, *cpf*, *data_nascimento*, *endereco* opcional, *anonimizado* e *data_cadastro*), abstendo-se do armazenamento de dados sensíveis supérfluos. Nas credenciais segregadas na entidade *UsuarioAuth* (tabela *usuario_auth*), as senhas jamais são persistidas em texto plano: o sistema aplica nativamente o *hash* unidirecional PBKDF2/SHA-256 (*PBKDF2PasswordHasher*) acrescido de *salt* criptográfico aleatório por meio do método *create_user()* do Django, mantendo a arquitetura de autenticação extensível ao algoritmo Argon2;
2. **Prova de Consentimento e Rastreabilidade (Art. 7º, I e Art. 8º, § 2º):** A LGPD elenca o fornecimento de consentimento pelo titular como hipótese autorizativa do tratamento (Art. 7º, I) e estabelece que essa manifestação deve ser livre, informada e inequívoca, cabendo ao controlador o ônus de comprovar sua obtenção legítima (Art. 8º, § 2º) (BRASIL, 2018). Para assegurar rastreabilidade jurídica auditável, o AppBus estrutura as tabelas *Documento_Legal* (que versiona os instrumentos *TERMOS_DE_USO* e *POLITICA_PRIVACIDADE* preservando o texto integral no atributo *conteudo*) e *Registro_Consentimento*, armazenando de forma imutável o aceite expresso do usuário (*consentimento_concedido*, em modalidade *opt-in*), a data e hora exatas da confirmação (*data_consentimento*), o endereço de rede da conexão (*ip_address*, capturado no servidor a partir de *request.META*) e a identificação do agente do navegador ou dispositivo (*user_agent*);

3. **Tratamento de Dados de Menores de Idade (Art. 14, § 1º):** O Art. 14 da LGPD dispõe que o tratamento de dados pessoais de crianças e de adolescentes deve ser realizado em seu melhor interesse, exigindo o § 1º consentimento específico e em destaque prestado por pelo menos um dos pais ou pelo responsável legal nos casos previstos em lei, em consonância com o Estatuto da Criança e do Adolescente — ECA (BRASIL, 2018). No AppBus, tanto o formulário reativo da interface PWA quanto a validação condicional de domínio no *backend* (método *Pessoa.is_menor_idade()* e regras de validação da API) verificam automaticamente a idade inferior a 18 anos com base no atributo *data_nascimento*, tornando obrigatório o preenchimento, a validação matemática e a gravação do atributo *responsavel_legal_cpf* na tabela *Registro_Consentimento*;
4. **Revogação e Mascaramento de Dados (Art. 12 e Art. 18, IV e VI):** O Art. 18 assegura ao titular o direito à anonimização (inciso IV) e à eliminação dos dados pessoais tratados com o seu consentimento (inciso VI), ao passo que o Art. 12 preceitua que os dados anonimizados não são considerados dados pessoais para os fins legais quando o processo técnico empregado impedir sua reversão por meios razoáveis (BRASIL, 2018). Diante da solicitação de exclusão de conta pelo discente, o sistema adota a política *No Hard Deletes* — abstendo-se da execução física do comando *DELETE* no banco de dados — por meio do método transacional *Pessoa.anonimizar()*. Essa rotina executa o mascaramento irreversível em três camadas: (i) na entidade *Pessoa*, sobrescreve *nome* para "Usuário Anonimizado", anula *cpf* e *data_nascimento*, limpa a referência residencial em *Pessoa.endereco* e define *Pessoa.anonimizado* como verdadeiro; (ii) na entidade *UsuarioAuth*, revoga o acesso definindo *ativo* como falso, inutiliza criptograficamente o segredo de autenticação via *set_unusable_password()*, substitui o correio eletrônico pelo padrão *anonimizado_[id]@appbus.local* e registra automaticamente o instante da modificação em *UsuarioAuth.data_atualizacao*; e (iii) na entidade *Aluno*, mascara o registro acadêmico *matricula* no padrão *ANON_[id]*. Dessa forma, desassocia-se definitivamente a identidade civil do titular, preservando-se a integridade referencial e os registros numéricos não identificáveis de viagens na tabela *Embarque* (*on_delete=models.PROTECT*) para fins de prestação de contas e auditoria pública municipal.

3. MATERIAIS E MÉTODOS

Para o desenvolvimento do sistema proposto, adotou-se uma abordagem fundamentada nos princípios da *Design Science Research* (DSR), conforme Hevner et al. (2004), metodologia voltada à concepção, ao desenvolvimento e à avaliação de artefatos destinados à solução de problemas práticos na área de Sistemas de Informação. Essa abordagem permitiu organizar o desenvolvimento do AppBus de maneira sistemática, relacionando a identificação do problema às etapas de planejamento, construção, aperfeiçoamento e avaliação do artefato.

Quanto à natureza, a pesquisa caracteriza-se como aplicada, pois busca desenvolver uma solução computacional direcionada a um problema prático relacionado à gestão do transporte universitário intermunicipal. Quanto aos objetivos, apresenta caráter exploratório, uma vez que procura ampliar a compreensão do problema, identificar suas características e levantar informações que contribuam para a definição das funcionalidades e dos requisitos do sistema (GIL, 2019; PIOVESAN; TEMPORINI, 1995).

A condução do trabalho foi organizada em quatro etapas principais: planejamento e revisão de literatura; definição dos critérios de qualidade; desenvolvimento e iteração; e avaliação e validação. Essas etapas são realizadas de forma integrada, permitindo que os conhecimentos obtidos durante a pesquisa orientem as decisões relacionadas à construção e ao aprimoramento do sistema.

Planejamento e Revisão de Literatura

Inicialmente, foram realizados o planejamento do projeto e o levantamento bibliográfico relacionado aos temas que fundamentam o desenvolvimento do sistema. Essa etapa teve como finalidade delimitar o problema do transporte universitário intermunicipal, compreender as dificuldades associadas à previsão da demanda e ao controle de embarque de passageiros, conhecer as características das soluções tecnológicas existentes e identificar os requisitos relacionados à proteção de dados pessoais.

A revisão de literatura serviu de base para a definição das características do sistema e para a fundamentação das decisões técnicas tomadas ao longo do projeto. Foram analisados trabalhos sobre mobilidade urbana, transporte universitário, previsão de demanda, sistemas de controle de embarque, aplicações web progressivas, arquitetura de serviços e proteção de dados. Conforme Gil (2019), a pesquisa exploratória possibilita maior familiaridade com o problema

estudado e contribui para seu melhor delineamento, o que a torna adequada para fundamentar a escolha da arquitetura web progressiva adotada no projeto.

Além do levantamento bibliográfico, foram observadas soluções comerciais voltadas ao transporte escolar e ao fretamento, como MeuEscolar, VanEscola e Clickvan, bem como trabalhos acadêmicos correlatos, entre eles os de Sousa, Barbosa e Hinz (2022), Pinto (2017), Dias (2023) e Moreira, Spagnol e Barbosa (2024). Essa análise buscou identificar funcionalidades, limitações e características relevantes para o AppBus. Também foi realizado o levantamento dos requisitos funcionais e não funcionais do sistema, entendidos como a descrição dos serviços que o *software* deve oferecer e das restrições sob as quais deve operar (SOMMERVILLE, 2018). Dessa forma, o planejamento inicial estabeleceu um direcionamento para a construção do artefato, considerando as necessidades do contexto do transporte universitário intermunicipal e as exigências da Lei Geral de Proteção de Dados Pessoais (BRASIL, 2018).

Definição dos Critérios de Qualidade

Após o levantamento inicial, foram estabelecidos os critérios que servem de referência para o desenvolvimento e para a posterior avaliação do sistema. Eles foram definidos para assegurar que o artefato não apenas atenda às funcionalidades previstas, mas também apresente características adequadas de uso, segurança, confiabilidade e conformidade legal.

Entre os principais critérios considerados, destacam-se:

- **Adequação funcional:** verifica se o sistema atende às necessidades de estudantes, motoristas e gestores, em especial o registro da intenção de viagem ("Vou/Não vou"), a validação do embarque por leitura de *QR Code* e a liberação manual nominal de contingência;
- **Confiabilidade:** relacionada à consistência das informações processadas, como a contagem das intenções de viagem que gera a estimativa de ocupação, e à disponibilidade do painel do motorista em cenários de sinal móvel instável, por meio da abordagem *offline-first*;
- **Usabilidade:** relacionada à facilidade de aprendizado, compreensão e utilização das funcionalidades por públicos com diferentes níveis de familiaridade com a tecnologia;

- **Portabilidade:** relacionada ao funcionamento da interface *Progressive Web App* (PWA) em diferentes dispositivos móveis e navegadores, inclusive em aparelhos de menor capacidade;
- **Segurança e Conformidade:** voltadas à proteção das informações e dos mecanismos de acesso, incluindo a separação lógica dos dados entre diferentes clientes, e à observância das normas legais aplicáveis ao tratamento dos dados pessoais.

No aspecto de usabilidade, foram considerados os conceitos da ISO 9241-11:2018, que define usabilidade como a medida em que um sistema pode ser usado por usuários específicos para atingir objetivos específicos com eficácia, eficiência e satisfação em um contexto de uso determinado (ISO, 2018). Também foi considerada a aplicação da *System Usability Scale* (SUS), proposta por Brooke (1996), como instrumento de apoio à avaliação da percepção dos usuários em relação ao sistema.

No aspecto de proteção de dados, os critérios foram estabelecidos com base nos princípios e requisitos da LGPD, especialmente os relacionados à finalidade, à necessidade, à segurança e ao tratamento adequado dos dados pessoais (BRASIL, 2018; DONEDA, 2020). Nesse contexto, foram previstos a minimização da coleta de dados, o registro da prova de consentimento, o tratamento diferenciado de menores de idade e o fluxo de anonimização dos dados do titular que solicitar sua exclusão.

Desenvolvimento e Iteração

A etapa de desenvolvimento corresponde à construção progressiva do artefato computacional, seguindo os princípios iterativos da *Design Science Research* (HEVNER et al., 2004). O sistema é desenvolvido em ciclos voltados à construção do Produto Mínimo Viável (MVP), nos quais as funcionalidades são implementadas, analisadas e aperfeiçoadas de acordo com os requisitos definidos e com as necessidades identificadas ao longo do projeto.

Cada ciclo envolve a modelagem da estrutura do sistema, a implementação das funcionalidades, a integração entre seus componentes e a realização de testes. A arquitetura adotada é desacoplada: o *frontend*, desenvolvido como PWA, comunica-se com o *backend* por meio de uma API baseada no estilo arquitetural REST (FIELDING, 2000), o que permite evoluir cada camada de forma independente (BIØRN-HANSEN; MAJCHRZAK; GRØNLI, 2017). A divisão em ciclos possibilita identificar inconsistências, ajustar funcionalidades e realizar melhorias antes da consolidação do MVP.

A abordagem iterativa também permite que decisões técnicas sejam revistas durante a execução do projeto, mantendo o desenvolvimento alinhado aos objetivos estabelecidos. Dessa maneira, o sistema passa por um processo contínuo de refinamento, buscando maior estabilidade, organização e adequação às necessidades dos usuários e aos gargalos operacionais observados no transporte universitário.

Na aplicação prática ao projeto AppBus, os ciclos iterativos da *Design Science Research* foram operacionalizados em um roteiro de desenvolvimento dividido em quatro fases sequenciais voltadas à consolidação do Produto Mínimo Viável (MVP):

- **Fase 1 – Infraestrutura, Padrão Party/Role e Isolamento Multi-Tenant:** Compreendeu a configuração do ambiente conteinerizado via arquivo *docker-compose.yml*, orquestrando três serviços interconectados na rede *bridge appbus_net*: *db* (imagem *postgres:15-alpine*, contêiner *appbus_db*, porta 5432, com volume persistente *postgres_data*), *web* (imagem *python:3.11-slim*, contêiner *appbus_backend*, porta 8000, configurado com *python manage.py runserver 0.0.0.0:8000* no ambiente de desenvolvimento para permitir *hot-reload* via *bind mount* do diretório *backend* no caminho interno *app*, mantendo o servidor WSGI *gunicorn* versão 21.2.0 ou superior instalado no *requirements.txt* para implantação em produção) e *frontend* (imagem *node:20-alpine*, contêiner *appbus_frontend*, porta 5173). No banco de dados e no ORM do Django, implementou-se a modelagem relacional sob o padrão *Party/Role*, vinculando o modelo customizado *Usuario_Auth* (*UsuarioAuth*) à entidade *Pessoa* (*Party*) e às extensões de papel (*Cliente*, *Instituicao*, *Administrador*, *Aluno* e *Motorista*), além das tabelas de endereçamento (*UF*, *Cidade*, *Bairro* e *Endereco*). A autenticação da API RESTful foi estruturada com *JSON Web Tokens* (*SimpleJWT*) e o isolamento lógico *multi-tenant* foi assegurado mediante a classe base *TenantBaseViewSet*, com sobrescrita dos métodos *get_queryset*, *perform_create* e *perform_update* para filtragem e vinculação estrita por *cliente_id*.
- **Fase 2 – Governança LGPD e Fluxo de Aprovação:** Envolveu a implementação das entidades de governança (*Documento_Legal* e *Registro_Consentimento*), o fluxo de autocadastro discente protegido por transações atômicas (*transaction.atomic*) com validação condicional de consentimento assistido para menores de 18 anos (*responsavel_legal_cpf*), a etapa de triagem e aprovação cadastral de estudantes pela gestão municipal (*TipoStatusAprovacao*) e o *endpoint* de autogestão de privacidade e anonimização de dados pessoais (*Pessoa.anonimizar*, realizando mascaramento

cadastral irreversível e inativação lógica sem remoção física via instrução *DELETE* no banco de dados).

- **Fase 3 – Operação de Viagens, Predição com Prioridade de Ciclo, Contingência Offline e Avisos:** Abrangeu a modelagem e a codificação das regras de negócio das entidades de operação logística e comunicação (*Modelo_Veiculo*, *Onibus*, *Rota*, *Rota_Instituicao*, *Viagem*, *Previsao_Viagem*, *Embarque* e *Aviso*). Foram construídos a rota HTTP *PATCH* de intenção de viagem ("Vou/Não vou") com cálculo de prioridade de ciclo e fila de espera para o retorno, a contagem de ocupação via agregação *COUNT*, os fluxos de *check-in* com trava anti-superlotação, o mural de avisos operacionais (*AvisoViewSet*) e o mecanismo de contingência *offline-first* do painel do motorista (*DriverPanelPage.vue*), equipado com lista de chamada manual e fila persistida no *localStorage* sob a chave *appbus_offline_checkins*, monitorando os eventos nativos *online* e *offline* (*window.navigator.onLine*) para sincronização automática em lote (*syncOfflineQueue*).
- **Fase 4 – Interface PWA e Experiência Mobile-First:** Concentrou-se na camada de apresentação e na experiência móvel utilizando Vue 3 (*Composition API*), Quasar Framework, Pinia, Vue Router, Axios e *vite-plugin-pwa* para configuração do *Service Worker* (via Google Workbox) e do *Web App Manifest*. Sob a abordagem *Mobile-First* e identidade visual de alto contraste (modo escuro com destaque em amarelo escolar), foram consolidados o painel administrativo web (aprovação de cadastros, controle de frota e rotas e geração do cartaz A4 do veículo contendo o *qr_code_uuid* renderizado em *elemento canvas HTML5* pela biblioteca *qrcode*, versão 1.5.4 ou compatível, por meio do método *QRCode.toCanvas* com nível de correção de erro *errorCorrectionLevel* definido como *H*), o mural de comunicados assíncronos (*MuralAvisosPage.vue*), a carteirinha digital estudantil, os cartões de confirmação rápida de embarque (ida e volta) com indicador dinâmico de prioridade de ciclo e fila de espera no retorno, o painel de consulta de consentimento e revogação de conta (LGPD) e a integração com a câmera traseira do *smartphone* pela biblioteca *html5-qrcode* (versão 2.3.8 ou compatível, classe *Html5Qrcode*, com suporte a lanterna e API nativa *BarcodeDetector*) para leitura óptica do *QR Code* do ônibus no momento do embarque, além dos ajustes finais de estabilidade e desempenho do MVP.

Avaliação e Validação

A quarta etapa do método compreende a verificação técnica e a avaliação de qualidade do Produto Mínimo Viável (MVP), visando constatar se o artefato construído atende aos requisitos funcionais e não funcionais especificados, em consonância com o ciclo avaliativo da *Design Science Research* (HEVNER et al., 2004). No âmbito de infraestrutura e persistência, o procedimento contempla testes funcionais sobre os *endpoints* da API RESTful e testes de concorrência voltados à simulação de múltiplos registros de embarque simultâneos, validando o controle de concorrência multiversão (*Multi-Version Concurrency Control* — MVCC) do PostgreSQL e a integridade das transações atômicas. Na camada de interface, realiza-se a auditoria automatizada da aplicação por meio da ferramenta Google Lighthouse (integrada ao Chrome DevTools), aferindo métricas de desempenho, boas práticas, operação do *Service Worker*, armazenamento em *cache* e critérios técnicos de instalabilidade do formato PWA. De forma complementar, a avaliação da interface considera os parâmetros de eficácia, eficiência e satisfação da norma ISO 9241-11:2018 (ISO, 2018) e a estrutura de mensuração da escala *System Usability Scale* — SUS (BROOKE, 1996) sobre os fluxos operacionais do sistema.

3.1. Python

O Python é uma linguagem de programação de alto nível, interpretada e de propósito geral, caracterizada por sua sintaxe legível e por um amplo ecossistema de bibliotecas (OLIPHANT, 2007). No AppBus, atua como a linguagem base do *backend* (executada sobre a imagem *python:3.11-slim*), sendo utilizada em conjunto com o *framework* Django para a codificação das regras de negócio, rotinas de anonimização cadastral, validações de embarque e manipulação de transações no banco de dados. A pilha de pacotes do *backend* definida no arquivo *requirements.txt* inclui, além do ecossistema Django, as bibliotecas *psycopg2-binary* (versão 2.9.9 ou superior), responsável pela comunicação nativa em linguagem C entre o ORM e o banco de dados PostgreSQL, *python-dotenv* (versão 1.0.0 ou superior), empregada em *settings.py* (*load_dotenv*) para o carregamento seguro de variáveis de ambiente a partir do arquivo *.env*, e *gunicorn* (versão 21.2.0 ou superior), servidor HTTP WSGI voltado à implantação (*deploy*) em ambiente de produção.

3.2. Django e Django REST Framework

O Django é um *framework* web escrito em Python que fornece os componentes estruturais para a construção rápida e segura de aplicações web, incluindo um mapeamento

objeto-relacional (*Object-Relational Mapping* — ORM) próprio (HOLOVATY; KAPLAN-MOSS, 2009). O Django REST Framework (DRF) consiste em um conjunto de ferramentas construído sobre o Django para o desenvolvimento de APIs web, disponibilizando serializadores (*Serializers*), visões baseadas em conjuntos (*ViewSets*) e roteadores (ENCODE OSS, 2026). No AppBus (com *Django* versão 4.2 ou superior, inferior à série 5.1, e *djangorestframework* versão 3.14.0 ou superior), o Modelo Entidade-Relacionamento (*Entity-Relationship Model* — ERM) sob o padrão *Party/Role* é traduzido diretamente para a camada de modelos (*models.py*) com suporte a transações atômicas (*transaction.atomic*) e *hashing* nativo de senhas via algoritmo *PBKDF2* (*PBKDF2PasswordHasher* com SHA-256). Na camada de governança e conformidade com a LGPD, a entidade *Registro_Consentimento* persiste de forma auditável os atributos *ip_address*, *user_agent*, *responsavel_legal_cpf*, *data_consentimento* e *consentimento_concedido*, extraindo o endereço IP e o agente de usuário diretamente dos cabeçalhos HTTP da requisição (*request.META*). Por sua vez, o DRF é empregado na construção dos *endpoints* por meio da classe base *TenantBaseViewSet* (com sobrescrita dos métodos *get_queryset*, *perform_create* e *perform_update*), que filtra e vincula compulsoriamente os registros pela chave *cliente_id*, impedindo o vazamento ou a injeção de dados entre diferentes municípios (*multi-tenant*).

3.3. API RESTful

O estilo arquitetural REST (*Representational State Transfer*), definido por Fielding (2000) para sistemas distribuídos, fundamenta-se em restrições como a separação entre cliente e servidor, comunicação sem estado (*stateless*), uso de *cache*, interface uniforme e organização em camadas. Uma interface de programação que implementa essas diretrizes é denominada RESTful (FIELDING, 2000). No AppBus, essa arquitetura viabiliza o desacoplamento entre as camadas do projeto: todas as regras de negócio do *backend* são expostas por meio de *endpoints* padronizados em formato JSON — protegidos por autenticação via *JSON Web Tokens* (*djangorestframework-simplejwt*, versão 5.3.0 ou superior) e controle de origens cruzadas via *django-cors-headers* (versão 4.3.0 ou superior), apoiados pelas bibliotecas de infraestrutura *psycopg2-binary* (versão 2.9.9 ou superior) e *python-dotenv* (versão 1.0.0 ou superior) — e consumidos de forma assíncrona pelo *frontend* através dos métodos HTTP (*GET*, *POST*, *PUT*, *PATCH* e *DELETE*).

3.4. PostgreSQL

O PostgreSQL é um Sistema Gerenciador de Banco de Dados Objeto-Relacional (SGBDOR) de código aberto, reconhecido pela conformidade transacional e pelo suporte rigoroso a chaves estrangeiras e integridade referencial (MOMJIAN, 2001). Para o gerenciamento de acessos simultâneos, utiliza o controle de concorrência multiversão (*Multi-Version Concurrency Control* — MVCC), técnica que mantém múltiplas versões dos registros para que operações concorrentes de leitura e escrita não se bloqueiem mutuamente (BERNSTEIN; GOODMAN, 1983). No AppBus, o PostgreSQL 15 (integrado ao Django via adaptador *psycopg2-binary*, versão 2.9.9 ou superior) armazena todas as entidades definidas no modelo ERM — entre elas *Usuario_Auth*, *Pessoa*, *Cliente*, *Instituicao*, *Administrador*, *Aluno*, *Motorista*, *Modelo_Veiculo*, *Onibus*, *Rota*, *Rota_Instituicao*, *Viagem*, *Previsao_Viagem*, *Embarque*, *Documento_Legal*, *Registro_Consentimento* e *Aviso* —, garantindo consistência transacional em conjunto com bloqueios pessimistas (*select_for_update*) mesmo durante picos simultâneos de leitura de *QR Code* no momento do embarque. Para assegurar a integridade semântica dos domínios no banco de dados e na camada de aplicação, as restrições de valores são padronizadas em *models.py* por meio de seis enumerações baseadas em *models.TextChoices*, compreendendo: *SentidoViagem* (*ida*, *volta*), *StatusViagem* (*agendada*, *em_andamento*, *concluida*, *cancelada*), *MetodoValidacao* (*qr_code_app*, *manual_motorista*), *TipoStatusAprovacao* (*PENDENTE*, *APROVADO*, *REJEITADO*), *TipoDocumentoLegal* (*TERMOS_DE_USO*, *POLITICA_PRIVACIDADE*) e *TipoAviso* (*GERAL*, *ATRASO*, *MANUTENCAO*, *URGENTE*).

3.5. Vue.js e Quasar Framework

O Vue.js é um *framework frontend* progressivo que adota o padrão arquitetural MVVM (*Model-View-ViewModel*) e o desenvolvimento modular baseado em componentes reativos, mostrando-se adequado à construção de aplicações de página única (*Single Page Applications* — SPA) desacopladas do *backend* (LI; ZHANG, 2021). Construído sobre o ecossistema Vue.js, o Quasar Framework oferece um catálogo padronizado de componentes visuais responsivos e recursos multiplataforma voltados à agilidade de desenvolvimento e à ergonomia em dispositivos móveis (DIAS, 2023). No AppBus, utiliza-se o Vue 3 (versão 3.4.21 ou compatível, com *Composition API* e *sintaxe script setup*) em conjunto com a ferramenta de construção Vite (versão 5.1.6 ou compatível), o Quasar Framework (versão 2.27.0 ou compatível), o gerenciador de estado Pinia (versão 2.1.7 ou compatível), o Vue Router (versão

4.3.0 ou compatível) e o cliente HTTP Axios (versão 1.6.8 ou compatível) para o desenvolvimento do painel administrativo web do gestor, do painel operacional de bordo do motorista, do mural de avisos e da interface móvel do estudante. Para os fluxos de validação óptica declarados no arquivo *package.json*, a camada *frontend* integra nominalmente duas bibliotecas especializadas: *qrcode* (versão 1.5.4 ou compatível), empregada em *AdminQRCodePage.vue* para a geração do cartaz A4 do veículo em *elemento canvas HTML5* (*QRCode.toCanvas* com nível alto de correção de erro *errorCorrectionLevel* definido como *H*), e *html5-qrcode* (versão 2.3.8 ou compatível), utilizada em *StudentQRScannerPage.vue* para a leitura óptica do *QR Code* pela câmera traseira do *smartphone* (*Html5Qrcode*, com suporte a acionamento de lanterna e aceleração nativa via API *BarcodeDetector*).

3.6. Progressive Web App (PWA)

Os *Progressive Web Apps* (PWAs) são aplicações web progressivas que podem ser instaladas diretamente a partir do navegador e oferecem uma experiência de uso próxima à de aplicativos nativos, sem depender da intermediação de lojas digitais (BIØRN-HANSEN; MAJCHRZAK; GRØNLI, 2017; TANDEL; JAMADAR, 2018). No AppBus, o *frontend* em Vue.js é integrado ao *plugin vite-plugin-pwa* (versão 0.19.8 ou compatível) para gerar o *Web App Manifest* (*manifest.webmanifest*) e registrar o *Service Worker* (construído com Google Workbox), o qual realiza o *cache* dos ativos estáticos da aplicação (*js*, *css*, *html*, *ico*, *png* e *svg*), economizando armazenamento nos *smartphones* dos estudantes. De forma complementar, na camada operacional do motorista (*DriverPanelPage.vue*), a fila de contingência *offline-first* armazena os *check-ins* manuais diretamente no *localStorage* do navegador sob a chave *appbus_offline_checkins*, monitorando os eventos nativos de conectividade *online* e *offline* (*window.navigator.onLine*) para executar a sincronização automática em lote (*syncOfflineQueue*) com a API assim que o sinal de rede é restabelecido.

3.7. Docker

O Docker é uma plataforma de conteinerização que realiza a virtualização no nível do sistema operacional, compartilhando o *kernel* da máquina hospedeira — diferentemente das máquinas virtuais tradicionais, que virtualizam a camada de *hardware*. Por meio de arquivos de especificação (*Dockerfile* e *docker-compose.yml*), o processo de construção das imagens e de comunicação entre serviços é descrito de forma declarativa e reprodutível, mantendo o ambiente consistente entre as etapas de desenvolvimento, teste e produção (MERKEL, 2014). No AppBus, o arquivo *docker-compose.yml* orquestra três serviços nomeados *db* (imagem

postgres:15-alpine, contêiner *appbus_db*, porta 5432, com volume persistente *postgres_data* e verificação de integridade via *pg_isready*), *web* (imagem *python:3.11-slim*, contêiner *appbus_backend*, porta 8000) e *frontend* (imagem *node:20-alpine*, contêiner *appbus_frontend*, porta 5173), conectados na rede *bridge appbus_net*. No ambiente de desenvolvimento conteinerizado, o serviço *web* (*appbus_backend*) executa o comando *python manage.py runserver 0.0.0.0:8000* para viabilizar o recarregamento automático de código (*hot-reload*) por meio de *bind mount* do diretório *backend* no caminho interno *app*, mantendo o servidor WSGI *gunicorn* (versão 21.2.0 ou superior) instalado no *requirements.txt* para a implantação (*deploy*) em ambiente de produção.

3.8. Google Antigravity IDE

O Google Antigravity é uma plataforma de desenvolvimento integrada e orientada a agentes (*agentic development platform*), que combina um editor de código assistido por inteligência artificial a agentes autônomos capazes de planejar, executar e verificar tarefas complexas de engenharia de *software* (GOOGLE, 2025). No projeto AppBus, constitui a ferramenta principal de desenvolvimento utilizada na escrita, refatoração e verificação dos componentes de *backend* e *frontend*.

4. APRESENTAÇÃO DO PROJETO

Este capítulo apresenta a especificação técnica e arquitetural do sistema AppBus, contemplando o levantamento de requisitos funcionais, não funcionais e regras de negócio, bem como a modelagem do sistema, a estrutura do banco de dados e as interfaces que compõem o Produto Mínimo Viável (MVP).

4.1. Requisitos de software

Conforme conceitua Sommerville (2018), os requisitos de um sistema estabelecem as descrições dos serviços que o *software* deve fornecer e as restrições operacionais sob as quais deve funcionar. Para assegurar a rastreabilidade e a verificação objetiva do MVP do AppBus, o levantamento foi estruturado em três categorias complementares: Requisitos Funcionais (RF), Requisitos Não Funcionais (RNF) e Regras de Negócio (RN).

4.1.1. Requisitos funcionais

Os Requisitos Funcionais descrevem as funcionalidades e os serviços disponibilizados pelo sistema para cada perfil de usuário (Aluno, Motorista e Administrador/Gestor). O Quadro 3 apresenta os 13 requisitos funcionais implementados no AppBus.

Quadro 3 – Requisitos Funcionais (RF) do sistema AppBus

Código	Requisito	Descrição Verificável	Perfil / Ator
RF01	Autenticação de Usuários Multi-perfil (JWT)	O sistema deve autenticar Alunos, Motoristas e Administradores mediante validação de credenciais na entidade *UsuarioAuth* (*POST /api/token/*), emitindo *tokens* JWT com *claims* de perfil (*role*, *cliente_id*, *pessoa_id*, *status_aprovacao*) e direcionando cada usuário ao painel correspondente ao seu papel (*Role*).	Todos
RF02	Auto-cadastro de Estudante (*Onboarding* LGPD)	O sistema deve permitir que o estudante realize seu auto-cadastro (*POST /api/auth/registro-aluno/*), coletando no formulário dados pessoais civis (*Pessoa*), vínculo municipal (*Cliente*) e institucional (*Instituicao*), credenciais de acesso (*UsuarioAuth*) e aceite expresso LGPD dos Termos de Uso vigentes (*Documento_Legal*), com suporte opcional a endereço estruturado (*Endereco*) na API.	Aluno

Código	Requisito	Descrição Verificável	Perfil / Ator
RF03	Registro de Prova de Consentimento	O sistema deve gravar automaticamente na tabela *Registro_Consentimento*, no ato do auto-cadastro, o vínculo com a versão vigente de *Documento_Legal*, o indicador booleano *consentimento_concedido*, o carimbo de data/hora (*data_consentimento*), o endereço IP de origem (*ip_address*), o identificador do navegador (*user_agent*) e, quando menor de 18 anos, o CPF do responsável legal (*responsavel_legal_cpf*).	Sistema / Aluno
RF04	Fila de Aprovação Cadastral	O sistema deve disponibilizar ao gestor municipal uma listagem filtrável de estudantes cadastrados (*PENDENTE*, *APROVADO*, *REJEITADO*), permitindo aprovar ou rejeitar a habilitação do aluno para utilização da frota e registrando o administrador avaliador (*avaliado_por*) e a data da decisão (*data_decisao*).	Administrador
RF05	Gestão de Frota e Rotas Municipais	O sistema deve permitir ao gestor municipal o cadastro, consulta, atualização e ativação/inativação da frota de veículos (*Onibus*) e das rotas municipais (*Rota* e o mapeamento ordenado de paradas em *RotaInstituicao*), apoiado pelos catálogos de modelos de veículos (*ModeloVeiculo*) e de instituições de ensino (*Instituicao*) na API e na administração do sistema.	Administrador
RF06	Geração e Emissão de *QR Code* Fixo do Veículo	O sistema deve gerar automaticamente um identificador único universal (*qr_code_uuid* no padrão UUIDv4) vinculado a cada ônibus cadastrado na tabela *Onibus*, disponibilizando a renderização e emissão de cartaz oficial A4 para impressão em PDF (*window.print()*) ou *download* em PNG.	Administrador
RF07	Declaração Dinâmica de Intenção de Viagem ("Vou/Não vou")	O sistema deve permitir que o estudante declare ou altere livremente sua intenção diária de embarque para os trechos de "Ida" (*vai_ida*) e de "Volta" (*vai_volta*) por meio de acionamento rápido (*toggle*), atualizando parcialmente (*PATCH*) o registro na tabela *PrevisaoViagem* e exibindo o status dinâmico de prioridade de assento (*GARANTIDA*, *CONFIRMADA* ou *FILA_ESPERA*) para o retorno.	Aluno
RF08	Abertura e Encerramento de Viagem	O sistema deve permitir que o motorista selecione a rota (*Rota*), o veículo (*Onibus*) e o sentido (*ida* ou *volta*) para instanciar uma nova viagem ativa na tabela *Viagem* com status *em_andamento*, bem como registrar seu encerramento ao término do percurso alterando o status para *concluida* (Concluída).	Motorista

Código	Requisito	Descrição Verificável	Perfil / Ator
RF09	Validação de Embarque via Leitura Óptica de *QR Code*	O sistema deve permitir que o estudante acione a câmera do *smartphone* pelo navegador para ler o *QR Code* fixado no veículo, validando o *qr_code_uuid* do ônibus, a viagem ativa (*em_andamento*), o vínculo e a aprovação do aluno e a disponibilidade de assentos para registrar a presença na tabela *Embarque* (*qr_code_app*), exibindo confirmação visual imediata em tela cheia (verde de sucesso ou vermelha de erro).	Aluno
RF10	Validação Manual de Embarque em Contingência *Offline*	O sistema deve disponibilizar no painel de bordo do motorista uma lista nominal com busca rápida dos alunos aprovados vinculados ao município, permitindo registrar manualmente o *check-in* na tabela *Embarque* (*manual_motorista*) e, em caso de ausência de conectividade móvel, persistir a fila *offline* de *check-ins* manuais em *localStorage* (*appbus_offline_checkins*) para sincronização automática quando a rede for restabelecida.	Motorista
RF11	Painel de Estimativa e Lotação em Tempo Real	O sistema deve exibir no painel do motorista, na visão do estudante e no *dashboard* administrativo indicadores consolidados via agregação SQL (*COUNT*), comparando a capacidade do veículo, o total de alunos previstos ("Vou"), o total de alunos efetivamente embarcados, as vagas livres e o percentual de ocupação na viagem ativa.	Motorista / Aluno / Administrador
RF12	Revogação de Consentimento e Anonimização de Conta	O sistema deve disponibilizar na tela de perfil do estudante a consulta aos dados cadastrais e à prova de consentimento, além da opção para revogar o consentimento e solicitar a exclusão da conta mediante confirmação expressa e validação da senha atual, acionando o *endpoint* de anonimização cadastral (*Pessoa.anonimizar()*).	Aluno
RF13	Mural de Avisos e Comunicados Assíncronos	O sistema deve disponibilizar um mural de avisos segmentado por município e rota, permitindo a gestores e motoristas publicar comunicados operacionais classificados (*GERAL*, *ATRASO*, *MANUTENCAO*, *URGENTE*), com suporte a respostas rápidas (*Quick Replies*) para motoristas em trânsito.	Todos

Fonte: Elaborado pelos autores (2026).

4.1.2. Requisitos não funcionais

Os Requisitos Não Funcionais definem as propriedades de qualidade, restrições arquitetônicas e padrões de desempenho, segurança e usabilidade sob os quais o sistema opera. O Quadro 4 detalha os requisitos não funcionais e seus respectivos critérios técnicos de verificação.

Quadro 4 – Requisitos Não Funcionais (RNF) do sistema AppBus

Código	Categoria	Descrição e Critério Técnico de Verificação
RNF01	Portabilidade e Leveza (PWA)	O *frontend* deve operar como *Progressive Web App* (PWA) utilizando Vue 3, Vite e *vite-plugin-pwa*, permitindo instalação direta pelo navegador com manifesto (*manifest.webmanifest*) válido e atingindo aprovação nos critérios técnicos de instalabilidade da auditoria Google Lighthouse.
RNF02	Confiabilidade e Operação *Offline-First*	O painel do motorista deve manter o *cache* de ativos estáticos da interface (*App Shell*) pelo *Service Worker* (Google Workbox) e persistir a fila de *check-in offline* em *localStorage* (*appbus_offline_checkins*), permitindo enfileirar embarques manuais durante perda de sinal móvel para sincronização automática em lote quando a conexão for restabelecida.
RNF03	Segurança e Isolamento *Multi-tenant*	O *backend* em *Django REST Framework* deve assegurar a segregação lógica de dados entre municípios (*TenantBaseViewSet*), sobrescrevendo o método *get_queryset* nas *ViewSets* para filtrar compulsoriamente todas as consultas pela chave *cliente_id* associada ao usuário autenticado e injetando o *tenant* autenticado em *perform_create* e *perform_update*.
RNF04	Segurança de Autenticação, *Hashing* e Auditoria	A comunicação entre *frontend* e API RESTful deve ser *stateless* e protegida por *JSON Web Tokens* (*SimpleJWT*) com *interceptors* HTTP (Axios) para injeção do cabeçalho *Bearer Token* e renovação transparente de sessão (*refresh token*). As senhas devem ser protegidas pelo algoritmo de *hashing* PBKDF2 (SHA-256) nativo do Django (*PBKDF2PasswordHasher*), e a captura de *ip_address* e *user_agent* para a prova de consentimento LGPD deve ocorrer diretamente no servidor via *request.META*.

Código	Categoria	Descrição e Critério Técnico de Verificação
RNF05	Integridade e Atomicidade Transacional	Operações de escrita que abrangem múltiplas tabelas (como o registro simultâneo em *UsuarioAuth*, *Endereco* opcional, *Pessoa*, *Aluno* e *Registro_Consentimento*, bem como a rotina *Pessoa.anonimizar()*) devem ser executadas sob transações atômicas (*transaction.atomic*), garantindo reversão integral (*rollback*) em caso de falha em qualquer etapa.
RNF06	Desempenho e Controle de Concorrência	O banco de dados PostgreSQL deve processar múltiplos registros de embarque simultâneos sem bloqueio mútuo entre leituras e escritas por meio do controle de concorrência multiversão (MVCC) combinado ao bloqueio pessimista (*select_for_update*), mantendo tempo de resposta da API inferior a 2 segundos nos fluxos de leitura de *QR Code*.
RNF07	Usabilidade e Ergonomia Visual (*Mobile-First*)	A interface deve seguir o padrão *Mobile-First* utilizando o Quasar Framework, com botões de acionamento amplos e tema de alto contraste com fundo escuro (*#0B0F19* / *#121212*) e amarelo escolar (*#F2C037*), atendendo aos critérios de eficácia, eficiência e satisfação da norma ISO 9241-11:2018.
RNF08	Padronização de Infraestrutura em Contêineres	Os três serviços declarados no *docker-compose.yml — db* (PostgreSQL 15 Alpine, contêiner *appbus_db*), *web* (contêiner *appbus_backend* com Django, operando via *runserver* em desenvolvimento e Gunicorn empacotado para produção) e *frontend* (contêiner *appbus_frontend* com Vue/Vite) — devem operar de forma orquestrada na rede *bridge appbus_net*, garantindo isolamento de dependências e paridade entre os ambientes de desenvolvimento e produção.

Fonte: Elaborado pelos autores (2026).

4.1.3. Regras de negócio

As Regras de Negócio estabelecem as políticas operacionais, restrições normativas e diretrizes legais (incluindo a conformidade com a LGPD) que governam o comportamento dos fluxos transacionais do sistema. O Quadro 5 apresenta as 10 regras de negócio implementadas no AppBus.

Quadro 5 – Regras de Negócio (RN) do sistema AppBus

Código	Regra de Negócio	Especificação Técnica e Restrição de Domínio
RN01	Segregação Estrita por Município (*Multi-tenant*)	Todo registro operacional de *Aluno*, *Motorista*, *Administrador*, *Instituicao*, *Onibus*, *Rota* e *Aviso* pertence estritamente a um único *Cliente* (*cliente_id*), sendo bloqueado qualquer acesso ou listagem cruzada entre prefeituras distintas.
RN02	Habilitação Condicionada à Aprovação Cadastral	O auto-cadastro discente cria o vínculo em *Aluno* com *status_aprovacao* definido como *PENDENTE*. Apenas estudantes com status *APROVADO* por um administrador municipal podem validar embarque óptico ou manual nas viagens da frota.
RN03	Consentimento Assistido para Menores de 18 Anos (LGPD Art. 14)	Caso a idade calculada a partir de *Pessoa.data_nascimento* seja inferior a 18 anos (*Pessoa.is_menor_idade()*), o sistema exige obrigatoriamente o preenchimento e a validação matemática do campo *responsavel_legal_cpf* na entidade *Registro_Consentimento*, somado ao aceite explícito do *Documento_Legal* vigente.

Código	Regra de Negócio	Especificação Técnica e Restrição de Domínio
RN04	Anonimização Irreversível sem Exclusão Física (*No Hard Deletes*)	Na revogação de consentimento e exclusão de conta, é vedado o comando *DELETE* físico de históricos operacionais (*Embarque* protegido por *on_delete=models.PROTECT*). O método *Pessoa.anonimizar()* aplica em transação atômica o mascaramento exato: *nome* alterado para "Usuário Anonimizado", *cpf*, *data_nascimento* e *endereco* definidos como *None*, *anonimizado* definido como *True*, *UsuarioAuth.ativo* definido como *False* (com *set_unusable_password()*), *email* mascarado como *anonimizado_[id]@appbus.local* e *Aluno.matricula* convertida para *ANON_[id]*.
RN05	Unicidade Diária de Intenção de Viagem	A entidade *PrevisaoViagem* impõe uma restrição de unicidade composta (*unique_together*) sobre a chave *(aluno, data)*, garantindo que cada estudante mantenha um único registro diário de predição atualizável parcialmente via *PATCH* para os trechos *vai_ida* e *vai_volta*.
RN06	Flexibilidade Operacional sem Trava Rígida de Horário	O sistema não bloqueia declarações de intenção de viagem nem registros de embarque por janelas fixas de relógio, exigindo apenas que a *Viagem* correspondente esteja com status *em_andamento* na data corrente.
RN07	Bloqueio de Duplicidade de Embarque na Mesma Viagem	Um mesmo estudante não pode ter mais de um registro na tabela *Embarque* para a mesma *Viagem* (seja via *qr_code_app* ou *manual_motorista*), utilizando bloqueio pessimista (*select_for_update*) para evitar registros duplicados em leituras concorrentes.

Código	Regra de Negócio	Especificação Técnica e Restrição de Domínio
RN08	Trava de Lotação Física Máxima (Anti-Superlotação)	O sistema deve bloquear novos registros de embarque (ópticos ou manuais) quando o total de passageiros embarcados atingir a capacidade máxima de assentos cadastrada para o veículo (*total_embarcados* ≥ *onibus.capacidade*).
RN09	Prioridade de Ciclo de Ida e Fila de Espera na Volta	Os alunos que confirmarem ou embarcarem no trecho de ida possuem assento prioritário assegurado para o retorno no mesmo dia (*GARANTIDA*); estudantes que solicitarem apenas o trecho de volta disputam as vagas remanescentes por ordem cronológica de confirmação (*CONFIRMADA*) e, caso a lotação seja atingida, são alocados automaticamente em Fila de Espera numerada (*FILA_ESPERA*).
RN10	Bloqueio de Viagens Simultâneas por Veículo	Um mesmo ônibus não pode possuir duas viagens com status *em_andamento* simultaneamente na mesma data.

Fonte: Elaborado pelos autores (2026).

4.2. Diagrama de casos de uso

O Diagrama de Casos de Uso representa as interações externas dos usuários (atores) com as funcionalidades essenciais do sistema, delimitando o escopo funcional da aplicação (SOMMERVILLE, 2018). No AppBus, a modelagem contempla três atores principais, correspondentes aos papéis (*Roles*) definidos na arquitetura:

1. **Aluno (Estudante):** Interage com a interface móvel PWA para autenticar-se no sistema (RF01), realizar o auto-cadastro com aceite dos Termos de Uso e prova de consentimento (RF02, RF03), declarar ou atualizar continuamente sua intenção diária de viagem para os trechos de ida e volta com visualização de prioridade e fila de espera (RF07), validar seu embarque escaneando o QR Code fixo do ônibus (RF09), acompanhar a ocupação da viagem ativa (RF11), gerenciar sua privacidade podendo revogar o consentimento e acionar a anonimização da conta (RF12) e consultar comunicados operacionais no mural de avisos do seu município e rota (RF13).

2. **Motorista:** Utiliza o painel operacional de bordo (com suporte *offline-first*) para autenticar-se (RF01), instanciar a abertura (*em_andamento*) e o encerramento (*concluida*) de viagens ativas (RF08), acompanhar em tempo real o contador comparativo entre alunos previstos, embarcados e vagas livres (RF11), realizar o *check-in* manual nominal em lista de contingência com fila *offline* em *localStorage* (RF10) e publicar avisos operacionais e respostas rápidas (*Quick Replies*) no mural municipal (RF13).
3. **Administrador (Gestor Municipal):** Opera o painel administrativo web do respectivo município (*cliente_id*) para autenticar-se (RF01), aprovar ou rejeitar cadastros pendentes de estudantes (RF04), gerenciar a frota de veículos (*Onibus*) e as rotas municipais (*Rota* e *RotaInstituicao*) apoiadas pelos catálogos de *Instituicao* e *ModeloVeiculo* (RF05), emitir o cartaz A4 com o QR Code fixo (*qr_code_uuid*) de cada ônibus para impressão (RF06), monitorar os indicadores operacionais e de ocupação das viagens (RF11) e publicar ou moderar comunicados no mural de avisos (RF13).

A Figura 5 apresenta o Diagrama de Casos de Uso do sistema AppBus.

Figura 5 – Diagrama de Casos de Uso do sistema AppBus

Inserir imagem do Diagrama de Casos de Uso aqui

Fonte: Elaborado pelos autores (2026).

4.3. Modelagem do banco de dados

A modelagem relacional do banco de dados no PostgreSQL foi estruturada sob o padrão arquitetural *Party/Role* e com suporte nativo à segregação *multi-tenant*, organizando as 21 tabelas do sistema em quatro domínios lógicos integrados:

- **Domínio de Autenticação e Governança LGPD:** A entidade *UsuarioAuth* (*usuario_auth*) substitui o modelo de autenticação padrão do *framework* Django para gerenciar as credenciais de acesso (*email* e *password*), permissões (*is_staff*, *is_superuser*), status de ativação (*ativo*) e carimbos temporais (*data_cadastro* e *data_atualizacao*). A conformidade legal com a Lei nº 13.709/2018 (BRASIL, 2018) é assegurada pelas tabelas *Documento_Legal* (*documento_legal*), que controla o versionamento dos Termos de Uso e Políticas de Privacidade (*TERMOS_DE_USO*, *POLITICA_PRIVACIDADE*), e *Registro_Consentimento* (*registro_consentimento*), que

armazena a prova digital imutável do aceite (*consentimento_concedido*, *data_consentimento*, *ip_address*, *user_agent* e, no caso de menores de 18 anos, *responsavel_legal_cpf*).

- **Domínio Party/Role e Localização Territorial:** A entidade central *Pessoa* (*pessoa*) mantém um relacionamento unívoco (1:1) com *UsuarioAuth* e concentra os atributos cadastrais civis (*nome*, *cpf*, *data_nascimento* e vínculo opcional com a hierarquia territorial normalizada composta por *UF*, *Cidade*, *Bairro* e *Endereco*), além do marcador booleano de controle de privacidade (*anonimizado*) e da coluna *data_cadastro*. A partir de *Pessoa*, estendem-se os papéis específicos de domínio (*Roles*) — *Administrador* (*administrador*), *Aluno* (*aluno*) e *Motorista* (*motorista*) —, todos obrigatoriamente vinculados a um registro da tabela *Cliente* (*cliente*), cuja chave estrangeira *cliente_id* governa o isolamento lógico *multi-tenant* em conjunto com a tabela *Instituicao* (*instituicao*).
- **Domínio de Frota e Rotas:** Compreende as tabelas *ModeloVeiculo* (*modelo_veiculo*) e *Onibus* (*onibus*, que armazena a placa, a capacidade de assentos e o identificador único universal *qr_code_uuid* impresso no QR Code fixo do veículo), além das tabelas *Rota* (*rota*) e *RotaInstituicao* (*rota_instituicao*), responsáveis pelo mapeamento dos itinerários municipais e da ordem de parada (*ordem_parada*) nas instituições de ensino atendidas em cada trajeto.
- **Domínio Operacional, Transacional e de Comunicação:** Reúne as tabelas de movimentação diária e comunicação do transporte. A entidade *Viagem* (*viagem*) registra a execução de um itinerário em determinada *data* e *sentido* (*ida*, *volta*), associando *Rota*, *Onibus* e *Motorista* e controlando o ciclo de vida operacional (*agendada*, *em_andamento*, *concluida*, *cancelada*). A tabela *PrevisaoViagem* (*previsao_viagem*) armazena as declarações dinâmicas de intenção ("Vou/Não vou") nos sentidos *vai_ida* e *vai_volta* por par (*aluno*, *data*), recebendo atualizações parciais (*PATCH*) dos estudantes. A tabela *Embarque* (*embarque*) consolida os registros de presença validados em tempo real (*timestamp*), discriminando o *metodo_validacao* utilizado (*qr_code_app* ou *manual_motorista*). Por fim, a entidade *Aviso* (*aviso*) armazena os comunicados operacionais assíncronos classificados por severidade (*GERAL*, *ATRASO*, *MANUTENCAO*, *URGENTE*) e segmentados por município, rota ou viagem.

O Quadro 6 sintetiza as 21 entidades do Modelo Entidade-Relacionamento (ERM) do AppBus, seus atributos sincronizados com o modelo físico (*backend/apps/core/models.py*) e suas respectivas responsabilidades.

Quadro 6 – Dicionário de entidades do Modelo Entidade-Relacionamento (ERM) do AppBus

Domínio	Entidade (*db_table*)	Principais Atributos e Chaves	Responsabilidade no Sistema
Autenticação e LGPD	*UsuarioAuth* (*usuario_auth*)	*id* (PK), *email* (Unique), *password* (Hash), *last_login*, *ativo* (Boolean), *is_staff*, *is_superuser*, *data_cadastro*, *data_atualizacao*	Gerenciamento de credenciais, autenticação JWT e inativação lógica de acesso sem exclusão física.
Autenticação e LGPD	*Documento_Legal* (*documento_legal*)	*id* (PK), *tipo* (*TERMOS_DE_USO*, *POLITICA_PRIVACIDADE*), *versao*, *conteudo* (Texto/Markdown), *data_publicacao*, *ativo*	Controle de versão e armazenamento do conteúdo integral dos Termos de Uso e Políticas de Privacidade.
Autenticação e LGPD	*Registro_Consentimento* (*registro_consentimento*)	*id* (PK), *pessoa_id* (FK), *documento_id* (FK), *consentimento_concedido* (Boolean), *responsavel_legal_cpf* (Nullable), *ip_address*, *user_agent*, *data_consentimento*	Armazenamento da prova digital imutável de consentimento e autorização de menores (Art. 8º e Art. 14 da LGPD).
Localização	*UF* (*uf*)	*id* (PK), *sigla* (Unique), *nome*	Cadastro normalizado das Unidades Federativas.
Localização	*Cidade* (*cidade*)	*id* (PK), *uf_id* (FK), *nome* (*unique_together*: *nome*, *uf*)	Cadastro de municípios vinculados à respectiva Unidade Federativa.
Localização	*Bairro* (*bairro*)	*id* (PK), *cidade_id* (FK), *nome* (*unique_together*: *nome*, *cidade*)	Cadastro de bairros vinculados a um município.
Localização	*Endereco* (*endereco*)	*id* (PK), *bairro_id* (FK), *logradouro*, *numero*, *complemento*, *cep*	Endereço estruturado e normalizado para pessoas e instituições de ensino.
Party / Role	*Cliente* (*cliente*)	*id* (PK), *cnpj* (Unique), *razao_social*, *nome_fantasia*, *ativo* (Boolean), *data_cadastro*	Entidade raiz do isolamento *multi-tenant* (prefeitura municipal ou organização gestora).

Domínio	Entidade (*db_table*)	Principais Atributos e Chaves	Responsabilidade no Sistema
Party / Role	*Pessoa* (*pessoa*)	*id* (PK), *usuario_id* (FK 1:1), *endereco_id* (FK Nullable), *nome*, *cpf* (Unique/Nullable), *data_nascimento*, *anonimizado* (Boolean), *data_cadastro*	Entidade base (*Party*) que centraliza dados pessoais civis e suporta mascaramento irreversível (*anonimizar()*).
Party / Role	*Aluno* (*aluno*)	*id* (PK), *pessoa_id* (FK 1:1), *cliente_id* (FK), *instituicao_id* (FK), *matricula*, *status_aprovacao* (*PENDENTE*, *APROVADO*, *REJEITADO*), *data_solicitacao*, *data_decisao*, *avaliado_por_id* (FK *Administrador*)	Papel (*Role*) do discente passageiro, controlando seu vínculo institucional e auditoria na fila de aprovação cadastral.
Party / Role	*Motorista* (*motorista*)	*id* (PK), *pessoa_id* (FK 1:1), *cliente_id* (FK), *cnh* (Unique), *categoria_cnh*, *ativo* (Boolean)	Papel (*Role*) do condutor habilitado para abrir ou concluir viagens e validar embarques manuais.
Party / Role	*Administrador* (*administrador*)	*id* (PK), *pessoa_id* (FK 1:1), *cliente_id* (FK), *cargo*, *ativo* (Boolean)	Papel (*Role*) do gestor municipal responsável pela administração da frota, rotas e aprovação de alunos.
Party / Role	*Instituicao* (*instituicao*)	*id* (PK), *cliente_id* (FK), *endereco_id* (FK), *nome*, *sigla*, *ativo* (Boolean)	Cadastro das instituições de ensino (universidades, faculdades e escolas técnicas) vinculadas ao município.
Frota e Rotas	*ModeloVeiculo* (*modelo_veiculo*)	*id* (PK), *marca*, *nome*, *ativo* (Boolean), *created_at*, *updated_at*	Catálogo padronizado de marcas e modelos de veículos rodoviários.
Frota e Rotas	*Onibus* (*onibus*)	*id* (PK), *cliente_id* (FK), *modelo_id* (FK), *placa* (Unique), *capacidade*, *qr_code_uuid* (UUIDv4 Unique), *ativo*, *created_at*, *updated_at*	Cadastro dos veículos da frota municipal, capacidade máxima de assentos e geração do UUID único do QR Code fixo.
Frota e Rotas	*Rota* (*rota*)	*id* (PK), *cliente_id* (FK), *nome*, *ativo*, *created_at*, *updated_at*	Definição dos itinerários de transporte universitário segregados por município.

Domínio	Entidade (*db_table*)	Principais Atributos e Chaves	Responsabilidade no Sistema
Frota e Rotas	*RotaInstituicao* (*rota_instituicao*)	*id* (PK), *rota_id* (FK), *instituicao_id* (FK), *ordem_parada*, *created_at*, *updated_at*	Tabela associativa que define as instituições atendidas por uma rota e a respectiva ordem sequencial de parada.
Operacional	*Viagem* (*viagem*)	*id* (PK), *rota_id* (FK), *onibus_id* (FK), *motorista_id* (FK), *sentido* (*ida*, *volta*), *status* (*agendada*, *em_andamento*, *concluida*, *cancelada*), *data*, *hora_inicio*, *hora_fim*, *ativo*, *created_at*, *updated_at*	Instanciação de uma viagem diária por sentido, controlando horários de abertura e fechamento, veículo e condutor alocados.
Operacional	*PrevisaoViagem* (*previsao_viagem*)	*id* (PK), *aluno_id* (FK), *data*, *vai_ida* (Boolean), *vai_volta* (Boolean), *ativo*, *created_at*, *updated_at*	Registro transacional único por (*aluno*, *data*) da intenção de viagem ("Vou/Não vou") atualizada via *PATCH*.
Operacional	*Embarque* (*embarque*)	*id* (PK), *viagem_id* (FK), *aluno_id* (FK), *metodo_validacao* (*qr_code_app*, *manual_motorista*), *timestamp*, *ativo*, *created_at*, *updated_at*	Registro imutável do *check-in* de presença do estudante na viagem (via leitura óptica de QR Code ou contingência manual).
Comunicação	*Aviso* (*aviso*)	*id* (UUID PK), *cliente_id* (FK), *autor_id* (FK *UsuarioAuth*), *rota_id* (FK Nullable), *viagem_id* (FK Nullable), *titulo*, *mensagem*, *tipo* (*GERAL*, *ATRASO*, *MANUTENCAO*, *URGENTE*), *ativo*, *criado_em*, *updated_at*	Mural de avisos e comunicados operacionais assíncronos emitidos por gestores e motoristas para os estudantes do município.

Fonte: Elaborado pelos autores (2026).

A Figura 6 apresenta o Diagrama Entidade-Relacionamento (DER) completo do banco de dados do sistema AppBus.

Figura 6 – Diagrama Entidade-Relacionamento (DER) do sistema AppBus

Inserir imagem do Diagrama Entidade-Relacionamento aqui

Fonte: Elaborado pelos autores (2026).

4.4. Arquitetura inicial do sistema

A arquitetura de software do AppBus foi projetada sob o modelo cliente-servidor desacoplado, dividida em três camadas independentes e orquestrada em ambiente conteinerizado via arquivo *docker-compose.yml* (conectadas pela rede *bridge* isolada *appbus_net*):

1. **Camada de Apresentação (Frontend PWA – serviço frontend, contêiner appbus_frontend):** Construída sobre a imagem *node:20-alpine* (porta *5173*) como uma *Single Page Application* (SPA) progressiva em Vue 3 (*Composition API*), empacotada via Vite e estilizada com componentes *Mobile-First* do Quasar Framework. O gerenciamento de estado reativo das sessões e perfis é centralizado na biblioteca Pinia, enquanto a comunicação assíncrona com o servidor é mediada pelo cliente HTTP Axios (dotado de *interceptors* para injeção do cabeçalho *Authorization: Bearer* e renovação transparente de *tokens* JWT). A camada progressiva (*vite-plugin-pwa*) implementa o modelo *App Shell* gerenciado por um *Service Worker* (Google Workbox) para o *cache* de ativos estáticos da aplicação, aliado ao armazenamento local em *localStorage* (*appbus_offline_checkins*) no painel do motorista, garantindo que o registro manual de embarques em contingência permaneça funcional mesmo em trechos rodoviários sem cobertura de internet móvel.
2. **Camada de Aplicação e Regras de Negócio (Backend API RESTful – serviço web, contêiner appbus_backend):** Executada sobre a imagem *python:3.11-slim* (porta *8000*) com o *framework* Python/Django (operando com *python manage.py runserver 0.0.0.0:8000* e *hot-reload* no ambiente de desenvolvimento conteinerizado e dispondo do servidor WSGI Gunicorn empacotado nas dependências para produção), exposta por meio do *Django REST Framework* (DRF) em conformidade com as restrições arquiteturais REST (FIELDING, 2000). Esta camada processa a autenticação *stateless* via SimpleJWT, aplica o isolamento lógico *multi-tenant* (*TenantBaseViewSet*) sobrescrevendo o método *get_queryset* nas *ViewSets* para filtragem estrita por *cliente_id*, coordena as transações atômicas (*transaction.atomic*) de cadastro e consentimento, executa o algoritmo de anonimização irreversível (*Pessoa.anonimizar()*) e processa as validações de embarque, controle de prioridade de ciclo, fila de espera e contagens de lotação (*COUNT*).
3. **Camada de Persistência (Banco de Dados Relacional – serviço db, contêiner appbus_db):** Implementada no SGBDOR PostgreSQL (*postgres:15-alpine*, porta

5432) com volume persistente (*postgres_data*), responsável pelo armazenamento íntegro das 21 entidades do modelo *Party/Role* e pelo processamento concorrente de escritas durante os horários de pico de embarque por meio do mecanismo *Multi-Version Concurrency Control* (MVCC) e bloqueios transacionais (*select_for_update*).

A Figura 7 ilustra a arquitetura inicial e a orquestração de contêineres do sistema AppBus.

Figura 7 – Arquitetura inicial e orquestração de contêineres do sistema AppBus

Inserir imagem do Diagrama de Arquitetura aqui

Fonte: Elaborado pelos autores (2026).

4.5. Protótipos de telas e interfaces do sistema

As interfaces do AppBus foram desenvolvidas sobre o Quasar Framework priorizando a ergonomia visual em trânsito e a facilidade de operação em dispositivos móveis de diferentes capacidades (*Mobile-First*), em conformidade com os critérios de eficácia, eficiência e satisfação da norma ISO 9241-11:2018 (ISO, 2018). Adotou-se uma identidade visual nativa em modo escuro (*Dark Mode*) de alto contraste — com fundo global em tom ultra-escuro *#0B0F19* (*Deep OLED Dark*) e superfícies de cartões e modais em *#121212* e *#121826* — combinada à cor de destaque Amarelo Escolar (*#F2C037* e *#FFC107*), cores semânticas operacionais (verde esmeralda *#10B981*, vermelho *#EF4444*, âmbar *#F59E0B* e azul informativo *#0EA5E9*) e tipografia sem serifa de alta legibilidade (*Inter*, *Segoe UI* e *Roboto*). Essa composição cromática favorece a leitura rápida tanto em ambientes noturnos de embarque quanto sob iluminação solar intensa, aliada a botões de área tátil ampliada (classe *.btn-bus-primary*) para minimizar erros de acionamento durante o deslocamento.

4.5.1. Telas de autenticação, auto-cadastro e consentimento LGPD

A interface inicial de autenticação (*LoginPage.vue*, rota */login*) unifica o ponto de entrada da aplicação e direciona automaticamente o usuário autenticado ao seu respectivo módulo funcional com base nas *claims* de perfil (*role*, *cliente_id*, *pessoa_id* e *status_aprovacao*) contidas no *token* JWT emitido por *POST /api/token/*. O cabeçalho exibe o emblema superior em destaque amarelo (*#F2C037*) com o ícone institucional de ônibus e o logotipo **AppBus** ("App" em branco e "Bus" em amarelo escolar). Logo abaixo, dispõe de um seletor visual de perfil estruturado em três abas (*Profile Chips*: *Aluno*, *Motorista* e *Gestor*), que

atualiza dinamicamente o quadro contextual descritivo ("Acesso do Estudante", "Painel do Motorista" ou "Gestão Municipal") e auxilia a navegação e homologação dos perfis. O cartão principal (*dark-card*) contém os campos de *E-mail Institucional ou Cadastrado* e *Sua Senha de Acesso* (equipado com botão de alternância de visibilidade *visibility* e *visibility_off*), o botão primário de submissão ("Entrar como [Aluno/Motorista/Gestor]"), o atalho inferior "Ainda não tem cadastro de Aluno? Solicite adesão" (que redireciona para a tela de auto-cadastro) e o rodapé de conformidade "Ambiente Seguro e Protegido conforme a LGPD". A Figura 8 ilustra a tela inicial de autenticação multi-perfil do sistema AppBus.

Figura 8 – Tela de autenticação multi-perfil do sistema AppBus

Inserir captura da Tela de Autenticação aqui

Fonte: Elaborado pelos autores (2026).

Por sua vez, a tela de auto-cadastro discente (*RegisterPage.vue*, rotas */registro* e */cadastro*) foi projetada sob o princípio da minimização de dados (Art. 6º, III da LGPD): para agilizar o *onboarding* móvel, a interface visual não exige o preenchimento de campos de endereço residencial pelo estudante, embora o *endpoint POST /api/auth/registro-aluno/* (*AlunoRegistroSerializer*) mantenha suporte opcional à estrutura de *Endereco* na camada de API. O formulário encontra-se organizado em três seções estruturadas dentro de um cartão escuro:

1. **Dados Pessoais e Titularidade:** Coleta os campos obrigatórios de *Nome Completo Civil*, *CPF* (com máscara numérica padronizada e algoritmo de validação matemática em tempo real dos dígitos verificadores da Receita Federal) e *Data de Nascimento*. Ao preencher a data de nascimento, o componente calcula reativamente a idade cronológica do estudante (*checkMaioridade*): caso a idade seja inferior a 18 anos, é exibido automaticamente, com transição suave (*slide-down*), um bloco em destaque amarelo/âmbar intitulado "Consentimento Assistido (LGPD Art. 14 / Menor de 18 Anos)", tornando obrigatório o preenchimento e a validação matemática do campo *CPF do Responsável Legal* (*responsavel_legal_cpf*), em estrito cumprimento ao Art. 14 da LGPD (BRASIL, 2018);
2. **Vínculo Municipal e Ensino:** Apresenta seletores dinâmicos (*q-select*) alimentados pela rota pública *GET /api/public/dados-adesao/*, compreendendo o seletor obrigatório de *Prefeitura / Município Concedente* (*cliente_id*), o seletor obrigatório e reativo de

Instituição de Ensino (*instituicao_id*, filtrado estritamente pelas faculdades e escolas técnicas vinculadas ao município selecionado) e o campo obrigatório de *Número de Matrícula Acadêmica* (*matricula*);

3. **Credenciais de Acesso e Privacidade:** Reúne os campos obrigatórios de *E-mail Institucional ou Pessoal*, *Senha de Acesso* (com validação de comprimento mínimo de oito caracteres e alternância de visibilidade) e *Confirmar Senha*, além da caixa de aceite expresso (*q-checkbox*) "Li e concordo expressamente com os Termos de Uso e Política de Privacidade", acompanhada de um modal (*q-dialog*) para leitura integral do texto jurídico vigente (*Documento_Legal* ativo retornado por *GET /api/documentos-legais/vigentes/*, exibindo versão e data de publicação).

A Figura 9 apresenta a interface de auto-cadastro discente com o bloco condicional de consentimento assistido da LGPD.

Figura 9 – Tela de auto-cadastro discente com validação condicional LGPD

Inserir captura da Tela de Auto-cadastro aqui

Fonte: Elaborado pelos autores (2026).

4.5.2. Painel administrativo do gestor

O módulo administrativo é voltado aos gestores municipais e coordenadores de transporte, oferecendo uma visão executiva consolidada e restrita aos dados vinculados ao seu *cliente_id*. A operação administrativa distribui-se em quatro telas integradas:

- **Painel Executivo (HomePage.vue, rotas / e /dashboard na visão de administrador):** Apresenta o cartão de boas-vindas com identificação do gestor e do município (*Município #ID*), um cartão em destaque para acesso rápido e pré-visualização do último comunicado publicado no **Mural de Avisos e Comunicados**, e três cartões de indicadores operacionais (KPIs) em tempo real: *Fila de Aprovação* (total de cadastros pendentes aguardando deferimento), *Frota Operacional* (total de ônibus ativos e cadastrados) e *Rotas e Linhas* (total de itinerários cadastrados). Abaixo dos indicadores, dispõe da seção "Ações Rápidas de Gestão Municipal" com botões de atalho direto para *Avaliar Fila de Alunos* (gerenciamento de cadastros), *Gerenciar Frota e Rotas*, *Imprimir Cartaz QR Code* e acesso ao *Mural de Avisos*, além do quadro inferior

"Conectividade do Sistema", que monitora em tempo real a saúde do *Backend* Django REST (porta *8000*), do banco PostgreSQL e do *Service Worker* PWA (Workbox);

- **Fila de Homologação Cadastral (AdminApprovalQueuePage.vue, rota /admin/aprovacoes):** Disponibiliza uma barra de pesquisa textual instantânea (por nome, CPF, matrícula ou instituição) combinada a um filtro segmentado (*q-btn-toggle*) com quatro visões: *Pendentes (N)*, *Aprovados*, *Rejeitados* e *Todos*. A listagem exibe cartões individuais para cada solicitação estudantil contendo Nome, selo visual (*badge*) colorido de status, CPF, Instituição de Ensino (*instituicao_nome*), Matrícula (*matricula*), E-mail, data e hora da solicitação (*data_solicitacao*) e identificação do administrador avaliador (*avaliado_por_nome*). Cada item conta com botão de inspeção detalhada em modal ("Ver Detalhes do Aluno") e botões de ação imediata **Aprovar** (*POST /api/alunos/[id]/aprovar/*) e **Rejeitar** (*POST /api/alunos/[id]/rejeitar/*), protegidos por diálogo modal de confirmação do Quasar;
- **Gestão de Frota e Rotas (AdminFleetPage.vue, rota /admin/frota):** Estrutura o gerenciamento logístico em duas abas superiores (*q-tabs*): **Frota de Ônibus** e **Rotas e Itinerários**. Na primeira aba, exibe a grade (*grid*) de veículos cadastrados com placa estilizada em padrão automotivo (*placa-badge*), selo interativo de status operacional (*Em Operação* ou *Inativo*, alternável via *PATCH /api/veiculos/onibus/[id]/*), marca e modelo, capacidade máxima de assentos, código *qr_code_uuid*, atalho para emissão do cartaz QR Code e modal de criação ("Cadastrar Novo Veículo", informando Placa, Modelo do Veículo oriundo de */api/veiculos/modelos/* e Capacidade de Assentos). Na segunda aba, lista as rotas municipais com seu status (*Ativa* ou *Desativada*), apresenta visualmente a sequência ordenada de paradas ("1º: Instituição A", "2º: Instituição B", mapeada em *RotaInstituicao*) e disponibiliza o modal de criação de novas rotas ("Cadastrar Nova Rota");
- **Emissão de Cartaz QR Code do Veículo (AdminQRCodePage.vue, rota /admin/qrcode):** Apresenta um seletor superior (*q-select*) para escolha do ônibus da frota e renderiza ao centro a pré-visualização fiel do **Cartaz Oficial em formato A4** (*#printable-poster*). O cartaz possui fundo branco para máximo contraste óptico, borda amarela (*#F2C037*), cabeçalho institucional ("SISTEMA APPBUS – TRANSPORTE UNIVERSITÁRIO – EMBARQUE DIGITAL VIA QR CODE"), quadro com Placa, Modelo e Capacidade Autorizada, o **QR Code central** renderizado em elemento *canvas* HTML5 pela biblioteca *qrcode* (*QRCode.toCanvas* com nível de correção de erros *H*)

contendo o *qr_code_uuid* do veículo, quadro explicativo com instruções em três passos para o estudante e rodapé de auditoria LGPD. No topo da tela, o gestor dispõe dos botões **Imprimir Cartaz (PDF)** — que aciona a janela nativa *window.print()* isolando exclusivamente o cartaz via regras CSS *media print* — e **Baixar PNG**, que exporta a imagem diretamente do elemento *canvas* HTML5 (*toDataURL*).

A Figura 10 apresenta a interface do painel administrativo do gestor municipal.

Figura 10 – Painel administrativo do gestor municipal

Inserir captura do Painel Administrativo do Gestor aqui

Fonte: Elaborado pelos autores (2026).

4.5.3. Painel operacional de bordo do motorista

O painel operacional de bordo (*DriverPanelPage.vue*, rota */motorista*, acessível também pelos atalhos rápidos do condutor em *HomePage.vue* juntamente com o Mural de Avisos) foi concebido com foco na simplicidade de condução e na resiliência em rodovias sem cobertura celular (*Offline-First*). O cabeçalho operacional exibe um selo reativo de conectividade (*Online* em verde com ícone *wifi* ou *Offline* em amarelo/laranja com ícone *wifi_off*), alimentado pelos eventos nativos do navegador (*window.navigator.onLine*). Quando há perda de sinal, a interface exibe um aviso informativo indicando que os embarques manuais serão salvos localmente; quando existem registros na fila local do *localStorage* (*appbus_offline_checkins*) e a conexão é restabelecida, surge uma faixa amarela de contingência indicando "N check-in(s) offline pendentes de sincronização" acompanhada do botão **Sincronizar Agora** (*syncOfflineQueue*).

A operação do painel alterna dinamicamente entre dois estados principais:

- **Estado 1 (Sem viagem ativa – Cartão "Iniciar Nova Viagem"):** Quando não há viagem em aberto para o condutor, a tela exibe o formulário de abertura contendo seletores obrigatórios (*q-select*) para a **Rota Municipal** e para o **Ônibus da Frota**, além de dois botões táteis de grandes dimensões para seleção obrigatória do **Sentido do Trajeto** (**IDA: Bairro para Faculdade** e **VOLTA: Faculdade para Bairro**) e o botão principal **INICIAR VIAGEM**, que aciona *POST /api/operacao/viagem/iniciar/* e instancia a viagem com status *em_andamento*;
- **Estado 2 (Viagem em andamento – Painel de Lotação e Check-in Manual):** Uma vez iniciada a viagem, o cartão superior destaca o selo verde pulsante **EM ANDAMENTO**,

o horário de abertura, o nome da rota, etiquetas (*chips*) com a placa do veículo e o sentido, e quatro cartões de indicadores (KPIs) em tempo real lado a lado: **Previstos** (total de alunos com intenção confirmada para o trecho), **Embarcados** (soma dos *check-ins* validados no servidor e na fila *offline* local), **Vagas Livres** (assentos restantes frente à capacidade do ônibus) e **Ocupação %** (percentual relativo de lotação). Abaixo dos indicadores, uma barra de progresso linear (*q-linear-progress*) reflete visualmente o nível de ocupação física (verde em ocupação regular, amarela/âmbar em faixa de atenção e vermelha ao atingir a lotação máxima), seguida pelo botão **CONCLUIR OPERAÇÃO DA VIAGEM** (*POST /api/operacao/viagem/[id]/concluir/*). Na parte inferior da tela, encontra-se a seção de **Check-in Manual de Passageiro (Contingência)**, equipada com barra de busca instantânea (*debounce* de 300 ms em *GET /api/operacao/alunos/busca/*) por nome, CPF ou matrícula. Os resultados exibem os estudantes aprovados do município com Nome, Matrícula, Instituição e **CPF mascarado (•••.123.•••-45)** em respeito à privacidade, permitindo validar o embarque com um toque no botão **Embarcar** (que muda imediatamente para a etiqueta verde **Embarcado**, operando tanto *online* via *POST /api/operacao/embarque/manual/* quanto em fila *offline* via *localStorage*).

A Figura 11 exibe o painel operacional de bordo do motorista e o mecanismo de contingência *offline*.

Figura 11 – Painel operacional de bordo do motorista e contingência offline

Inserir captura do Painel Operacional do Motorista aqui

Fonte: Elaborado pelos autores (2026).

4.5.4. Painel do estudante, predição, leitura óptica de QR Code e privacidade LGPD

O módulo móvel do estudante reúne os fluxos de identificação digital, predição diária de demanda, validação óptica de embarque e autogestão de privacidade em três interfaces principais. A primeira interface consiste no **Painel Principal do Estudante** (*StudentHomeSection.vue*, renderizado em *HomePage.vue* na rota */*), estruturado em quatro cartões funcionais:

1. **Carteirinha Estudantil Digital (AppBus – Passe Digital):** No topo da tela, apresenta o passe digital estilizado com avatar, Nome Completo do estudante, Matrícula, Instituição de Ensino, Município concedente, identificação "Ano Letivo 2026" e selo dinâmico de

situação cadastral (*APROVADO* em verde, *EM ANÁLISE* em amarelo ou *REJEITADO* em vermelho). Caso o cadastro ainda esteja *PENDENTE* ou tenha sido *REJEITADO*, é exibida logo abaixo uma faixa condicional amarela informando a restrição de embarque até a homologação pela prefeitura;

2. **Cartão "Ônibus Universitário Hoje":** Monitora em tempo real (*GET /api/operacao/viagem/ativa/*) se há viagem ativa no município. Quando um veículo está em percurso, exibe o selo verde pulsante **ÔNIBUS EM TRÂNSITO**, o horário de início, a rota, a placa, o sentido, o contador "Lotação: X / Y passageiros (Z vagas restantes)" e a barra de progresso linear; caso contrário, informa "Aguardando Início da Linha";
3. **Cartão "Minha Intenção de Viagem" ("Você vai hoje?"):** Exibe a data corrente, o indicador de sincronização automática ("Sincronizado", acompanhado de ícone visual de confirmação, ou "Salvando...") e dois botões táteis independentes de grande formato (*toggles*): **VOU NA IDA / NÃO VOU NA IDA** (Bairro para Campus) e **VOLTO NO ÔNIBUS / NÃO VOLTAREI** (Campus para Bairro). Qualquer toque dispara imediatamente uma requisição parcial *PATCH /api/operacao/predicao/* sem necessidade de botão adicional de submissão. Quando o estudante marca que utilizará o retorno (*vai_volta = true*), o cartão renderiza dinamicamente o **Indicador de Prioridade de Ciclo e Fila de Espera no Retorno** (RN09), exibindo um dos três estados calculados pelo *backend*: **Prioridade de Ciclo Garantida** (*GARANTIDA* com selo verde *ASSENTO ASSEGURADO*, para quem confirmou ou embarcou na ida), **Vaga Remanescente Confirmada** (*CONFIRMADA* com selo amarelo *VAGA DISPONÍVEL*, para quem solicitou apenas a volta dentro do limite de assentos livres) ou **Fila de Espera para a Volta** (*FILA_ESPERA* com selo âmbar pulsante *POSIÇÃO #N*, quando os assentos remanescentes se esgotam). Na base do cartão, o **Contador Comunitário** informa a previsão coletiva do dia ("Previsão coletiva hoje: X confirmados na ida e Y na volta");
4. **Cartão de Chamada "Embarque Óptico" e Acesso ao Perfil:** Destaca o botão amarelo de ação principal **LER QR CODE DO ÔNIBUS** (que conduz ao leitor óptico) e o botão inferior **Meu Perfil e Governança LGPD**.

A Figura 12 demonstra o painel principal do estudante com a carteirinha digital, os seletores de predição diária e o indicador de prioridade de ciclo.

Figura 12 – Painel principal do estudante com carteirinha digital e prioridade de ciclo

Inserir captura do Painel Principal do Estudante aqui

Fonte: Elaborado pelos autores (2026).

Em complemento ao painel inicial, a operação móvel de validação de presença e o exercício dos direitos do titular de dados pessoais concentram-se em duas interfaces dedicadas:

- **Leitor Óptico de QR Code (StudentQRScannerPage.vue, rota /aluno/qrcode):** Apresenta barra superior com botão de retorno, título "Embarque Óptico" e botão de acionamento de lanterna (*torch*: *flash_on* e *flash_off*) para embarques noturnos. O visor central de vídeo (*#appbus-qr-reader*, fundo escuro *#000000*) inicializa a câmera traseira do *smartphone* via biblioteca *html5-qrcode* (*Html5Qrcode* com *facingMode: environment* e área de leitura *250x250*), sobrepondo uma moldura reticular amarela (*260x260px*) com cantoneiras luminosas e linha de varredura animada (*laser-scanline*), além de contar com um painel expansível inferior ("Simulação de Testes / Entrada Manual") para validação em computadores desprovidos de câmera. Ao decodificar o *qr_code_uuid* e submeter *POST /api/operacao/embarque/qrcode/*, o sistema exibe telas cheias de retorno visual de alto contraste: **Tela Verde** (*#052E16* – **EMBARQUE CONFIRMADO!**), apresentando ícone pulsante de confirmação e o resumo auditável da passagem (Linha/Rota, Placa do Veículo, Nome do Aluno, Matrícula, Data/Hora e método *QR Code Óptico (App)*), ou **Tela Vermelha** (*#450A0A* – **EMBARQUE NÃO AUTORIZADO**), com animação de alerta (*shake*) e a descrição exata do impedimento (como embarque já realizado na viagem, cadastro pendente ou lotação máxima atingida);
- **Perfil do Estudante e Governança LGPD (StudentProfilePage.vue, rota /aluno/perfil):** Consome o *endpoint GET /api/auth/perfil/* para exibir com transparência (Art. 18, II da LGPD) os **Dados Cadastrais do Titular** (Nome Completo, CPF mascarado **•••.•••.•••-••**, Data de Nascimento, Instituição de Ensino e Prefeitura) e o cartão **Consentimento e Privacidade (LGPD)**, que documenta a base legal do tratamento (Execução de Políticas Públicas – Art. 7º, V da LGPD) e exibe o comprovante digital do aceite contendo a versão do *Documento_Legal*, a data e hora do consentimento (*data_consentimento*) e o endereço IP de origem registrado. Na seção inferior **Gerenciar Privacidade da Conta** (Art. 18, VI da LGPD), disponibiliza o botão **Revogar Consentimento e Excluir Minha Conta**, cujo acionamento abre um modal

crítico de segurança em duas etapas: exige a marcação expressa da caixa de ciência da irreversibilidade da ação e a digitação da **senha atual** do titular para disparar *POST /api/auth/solicitar-exclusao/*, executando a rotina atômica *Pessoa.anonimizar()* (*No Hard Deletes*) e encerrando a sessão.

A Figura 13 apresenta as interfaces do leitor óptico de QR Code e do painel de autogestão de privacidade LGPD.

Figura 13 – Leitor óptico de QR Code e painel de autogestão de privacidade LGPD

Inserir captura do Leitor Óptico de QR Code e Painel de Privacidade aqui

Fonte: Elaborado pelos autores (2026).

4.5.5. Mural de avisos e comunicados assíncronos

Para substituir a comunicação informal e fragmentada em aplicativos de mensagens, o sistema disponibiliza o **Mural de Avisos e Comunicados Assíncronos** (*MuralAvisosPage.vue*, rota */mural*), acessível a todos os perfis autenticados do município (*GET /api/avisos/*). Na parte superior, a interface oferece abas horizontais (*q-tabs*) para filtragem rápida por nível de severidade (**Todos**, **Urgentes**, **Atrasos**, **Manutenção** e **Gerais**) combinadas a um seletor dinâmico de **Filtro por Rota** (*Filtrar por Rota*), permitindo ao estudante visualizar tanto comunicados globais da prefeitura quanto alertas específicos da sua linha.

O *feed* central apresenta os comunicados em cartões estilizados (*aviso-card*) dotados de borda lateral esquerda colorida semanticamente conforme a criticidade do evento: vermelha (*#EF4444*) para **URGENTE**, amarela/âmbar (*#F59E0B*) para **ATRASO**, azul (*#0EA5E9*) para **MANUTENCAO** e amarela primária (*#F2C037*) para **GERAL**. Cada cartão exibe o nome do autor, o selo do seu perfil operacional (*Motorista* ou *Administrador*), a indicação da rota vinculada (*Linha: [rota_nome]* ou *Geral (Todas as Linhas)*), o carimbo de data e hora da publicação (*criado_em*), o título em destaque, o corpo da mensagem e, para o próprio autor ou gestor municipal, o botão "Inativar Comunicado" (*DELETE /api/avisos/[id]/*).

Para os perfis de **Gestor** e **Motorista**, a tela habilita o botão superior "Novo" e um botão de ação flutuante (*Floating Action Button* – FAB) no canto inferior direito, que abrem o modal **Novo Comunicado** (*POST /api/avisos/*). Visando à segurança viária e à redução drástica do tempo de digitação pelo condutor em operação, o modal incorpora exclusivamente para o perfil **Motorista** uma barra de **Respostas Rápidas** (*Quick Replies*) de um toque acompanhadas de

ícones visuais semânticos de alerta e status operacional ("Atraso 10m", "Atraso 20m", "Embarque Liberado", "Veículo Reserva" e "Trânsito Intenso"). Ao tocar em qualquer atalho rápido, o sistema preenche automaticamente o título, a mensagem padronizada e a classificação de severidade correspondente, restando ao motorista apenas confirmar o disparo instantâneo para os estudantes do município.

A Figura 14 ilustra a interface do Mural de Avisos e o modal de publicação com respostas rápidas.

Figura 14 – Mural de avisos e comunicados assíncronos com respostas rápidas

Inserir captura do Mural de Avisos e Comunicados aqui

Fonte: Elaborado pelos autores (2026).

5. CONSIDERAÇÕES FINAIS

O presente Trabalho de Conclusão de Curso teve como objetivo geral desenvolver o **AppBus**, um sistema computacional no formato *Progressive Web App* (PWA) integrado a uma API RESTful para a gestão preditiva de frotas e o controle nominal de embarque no transporte universitário intermunicipal. Diante dos gargalos históricos enfrentados por prefeituras e associações estudantis — caracterizados pela imprevisibilidade diária da demanda, episódios recorrentes de superlotação ou ociosidade veicular, controles manuais em listas de papel ou grupos informais de mensagens e ausência de adequação à Lei Geral de Proteção de Dados Pessoais (Lei nº 13.709/2018) —, a pesquisa aplicou o rigor metodológico da *Design Science Research* (DSR) para conceber, construir e validar um Produto Mínimo Viável (MVP) escalável, resiliente e juridicamente aderente à administração pública.

A síntese dos resultados obtidos demonstra que o objetivo geral e os cinco objetivos específicos delineados na investigação foram integralmente alcançados. No tocante ao primeiro objetivo específico, voltado à engenharia de requisitos e à modelagem das regras de domínio do transporte estudantil, foram mapeados, especificados e implementados **13 Requisitos Funcionais** (RF01 a RF13), **8 Requisitos Não Funcionais** (RNF01 a RNF08) e **10 Regras de Negócio** (RN01 a RN10), assegurando rastreabilidade completa entre as demandas operacionais de gestores, motoristas e estudantes e os módulos entregues no artefato de software.

Em atendimento ao segundo objetivo específico, projetou-se e consolidou-se uma arquitetura cliente-servidor desacoplada em três camadas, composta por *frontend* reativo em Vue.js 3 e Quasar Framework empacotado como PWA, *backend* em Python com Django REST Framework (DRF) sob autenticação *stateless* via JSON Web Tokens (JWT) e camada de persistência relacional no SGBDOR PostgreSQL, integralmente orquestrada em contêineres Docker. A modelagem relacional de **21 entidades**, estruturada sob o padrão arquitetural *Party/Role* — centralizado na entidade *Pessoa* e especializado nos papéis *Aluno*, *Motorista* e *Administrador* — e combinada ao mecanismo nativo de isolamento lógico *multi-tenant* filtrado compulsoriamente pelo vínculo municipal (*cliente_id*), garantiu que múltiplas prefeituras operem sobre a mesma infraestrutura computacional sem risco de vazamento ou contaminação cruzada de dados.

Quanto ao terceiro objetivo específico, a implementação do fluxo operacional híbrido eliminou a assimetria de informação entre passageiros, condutores e coordenadores de frota por meio de quatro frentes tecnológicas integradas: (i) a predição diária de demanda ("Vou/Não

vou") com acionamento independente para ida e volta, regida pelo algoritmo de **Prioridade de Ciclo** no retorno, que assegura o assento de volta aos discentes que realizaram o trecho de ida, aloca de forma transparente as vagas remanescentes e organiza automaticamente a **Fila de Espera** cronológica em horários de pico; (ii) a validação óptica dinâmica de embarque via **QR Code**, confrontando em tempo real a leitura da câmera do *smartphone* com a viagem ativa, a homologação cadastral do discente e a trava física anti-superlotação apoiada pelo controle de concorrência multiversão (MVCC) e bloqueio pessimista (*select_for_update*) no banco de dados; (iii) a contingência *offline-first* no painel de bordo do motorista, amparada por *Service Worker* (Google Workbox) e armazenamento persistente da fila de embarques manuais em *localStorage*, garantindo continuidade operacional em trechos rodoviários desprovidos de cobertura celular com sincronização automática em lote; e (iv) o **Mural de Avisos** assíncrono segmentado por município e rota, dotado de respostas rápidas (*Quick Replies*) de um toque para preservar a segurança viária na comunicação de bordo.

Em cumprimento ao quarto objetivo específico, o sistema incorporou nativamente os preceitos de *Privacy by Design* exigidos pela **LGPD** (BRASIL, 2018). A arquitetura assegurou a proteção criptográfica de credenciais mediante o algoritmo de *hashing* **PBKDF2/SHA-256**, a minimização de dados coletados no *onboarding* estudantil, a gravação imutável da prova digital de consentimento — registrando no servidor a versão vigente do documento legal, o carimbo temporal, o endereço IP de origem e o identificador do navegador (*user-agent*) — e a validação condicional obrigatória de consentimento assistido para estudantes **menores de 18 anos** mediante verificação matemática do CPF do responsável legal (Art. 14 da LGPD). Ademais, a rotina de **anonimização transacional irreversível** sem exclusões físicas (*no hard deletes*) conciliou o direito de revogação e eliminação de dados pessoais pelo titular (Art. 18, VI) com o dever de integridade auditável e estatística dos históricos de embarque pela gestão pública municipal.

No que concerne ao quinto objetivo específico, o MVP do AppBus foi consolidado sob diretrizes ergonômicas *Mobile-First*, identidade visual em alto contraste (*Dark Mode* com destaque em amarelo escolar) e áreas táteis ampliadas para uso em deslocamento, encontrando-se estruturado e pronto para a aplicação de avaliação empírica junto aos usuários reais. O protocolo avaliativo fundamenta-se nas métricas de eficácia, eficiência e satisfação estabelecidas pela norma internacional **ISO 9241-11:2018** (ISO, 2018), aliadas à aferição psicométrica padronizada de usabilidade por meio da escala *System Usability Scale* — **SUS** (BROOKE, 1996).

Por fim, como desdobramentos científicos e tecnológicos para trabalhos futuros visando à evolução contínua da plataforma AppBus, propõem-se quatro frentes principais: (i) a incorporação de telemetria GPS em tempo real via protocolo *WebSockets* para exibição contínua da geolocalização dos ônibus em mapa interativo aos estudantes e gestores; (ii) o treinamento e a aplicação de algoritmos de *Machine Learning* sobre as séries históricas de intenção e embarque efetivo para predição automatizada de taxas de abstenção (*no-show*) e apoio ao dimensionamento plurianual de frotas; (iii) a integração direta via API com as secretarias acadêmicas das instituições de ensino conveniadas para validação cadastral e renovação semestral automatizada do vínculo de matrícula; e (iv) a implementação de notificações *Web Push* nativas por intermédio da *Push API* do *Service Worker*, permitindo alertar instantaneamente os discentes promovidos na Fila de Espera do retorno ou diante da emissão de comunicados operacionais urgentes.

REFERÊNCIAS

BANERJEE, Nilotpal; MORTON, Alec; AKARTUNALI, Kerem. Passenger demand forecasting in scheduled transportation. **European Journal of Operational Research**, v. 286, n. 3, p. 797-810, 2020.

BERNSTEIN, Philip A.; GOODMAN, Nathan. Multiversion concurrency control: theory and algorithms. **ACM Transactions on Database Systems**, v. 8, n. 4, p. 465-483, 1983.

BIØRN-HANSEN, Andreas; MAJCHRZAK, Tim A.; GRØNLI, Tor-Morten. Progressive Web Apps: The Possible Web-native Unifier for Mobile Development. In: INTERNATIONAL CONFERENCE ON WEB INFORMATION SYSTEMS AND TECHNOLOGIES (WEBIST), 13., 2017, Porto. **Anais** [...]. Porto: SciTePress, 2017. p. 344-351.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). **Diário Oficial da União**, Brasília, DF, 15 ago. 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709compilado.htm. Acesso em: 20 set. 2026.

BROOKE, John. SUS: A Quick and Dirty Usability Scale. In: JORDAN, Patrick W.; THOMAS, Bruce; WEERDMEESTER, Bernard A.; MCCLELLAND, Ian L. (ed.). **Usability Evaluation in Industry**. London: Taylor & Francis, 1996. p. 189-194.

CARVALHO, Carlos Henrique Ribeiro de; BALBIM, Renato Nunes; PEREIRA, Rafael; CORREIA NETO, Vicente. Mobilidade urbana no Brasil: principais estudos produzidos pelo Ipea nos últimos quinze anos. **Boletim Regional, Urbano e Ambiental**, n. 30, 2023.

CLICKVAN. **Clickvan**: software de gestão para transporte escolar e frotas. [S. l.], 2026. Disponível em: https://clickvan.app/. Acesso em: 20 set. 2026.

DIAS, Emilly Nascimento. **Sistema de Monitoramento de Acesso**: Fluxo de Discentes no Campus IFFar Uruguaiana. 2023. 52 f. Trabalho de Conclusão de Curso (Técnico em Informática Integrado ao Ensino Médio) – Instituto Federal de Educação, Ciência e Tecnologia Farroupilha, Uruguaiana, 2023.

DONEDA, Danilo. **Da privacidade à proteção de dados pessoais**: fundamentos da Lei Geral de Proteção de Dados. 2. ed. São Paulo: Thomson Reuters Brasil, 2020.

ENCODE OSS. **Django REST framework**. [S. l.], [2026]. Disponível em: https://www.django-rest-framework.org/. Acesso em: 15 out. 2026.

FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures**. 2000. 162 f. Tese (Doutorado em Ciência da Computação e Informação) – University of California, Irvine, 2000.

FRANCO, Ana Maria de Paiva; CUNHA, Sarah. Perfil socioeconômico dos graduandos das IFES. **Boletim FUMARC**, 2014.

GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. 6. ed. São Paulo: Atlas, 2019.

GOOGLE. Build with Google Antigravity, our new agentic development platform. **Google Developers Blog**, 20 nov. 2025. Disponível em: https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/. Acesso em: 15 out. 2026.

HAMAD, Khaled; OBAID, Lubna. Tour-based travel demand forecasting model for a university campus. **Transport Policy**, v. 117, p. 118-137, 2022.

HEVNER, Alan R.; MARCH, Salvatore T.; PARK, Jinsoo; RAM, Sudha. Design Science in Information Systems Research. **MIS Quarterly**, v. 28, n. 1, p. 75-105, 2004.

HOLOVATY, Adrian; KAPLAN-MOSS, Jacob. **The definitive guide to Django**: web development done right. 2. ed. Berkeley: Apress, 2009.

ISO — INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 9241-11:2018**: Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts. Geneva: ISO, 2018.

KNEIB, Erika Cristine. Transporte público coletivo: dos desafios globais ao panorama brasileiro. **Revista de Direito da Cidade**, v. 14, n. 2, p. 794-819, 2022.

LI, Nian; ZHANG, Bo. The research on single page application front-end development based on Vue. **Journal of Physics**: Conference Series, v. 1883, n. 1, p. 012030, 2021. DOI: 10.1088/1742-6596/1883/1/012030.

MERKEL, Dirk. Docker: lightweight Linux containers for consistent development and deployment. **Linux Journal**, 19 maio 2014. Disponível em: https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment. Acesso em: 15 out. 2026.

MEUESCOLAR. **MeuEscolar**: aplicativo para gestão de transporte escolar. [S. l.], 2026. Disponível em: https://meuescolarapp.com.br/. Acesso em: 20 set. 2026.

MOMJIAN, Bruce. **PostgreSQL**: introduction and concepts. Boston: Addison-Wesley, 2001.

MOREIRA, Marcos Vinícius Simionato; SPAGNOL, Michell Henrique da Silva; BARBOSA, Otília Donato. AcadBus: aplicativo de controle para transporte de acadêmicos. In: SEMINÁRIO INTEGRADO DE ENSINO, PESQUISA E EXTENSÃO (SIEPE), 17., 2024, São Miguel do Oeste. **Anais** [...]. São Miguel do Oeste: Unoesc, 2024.

OLIPHANT, Travis E. Python for scientific computing. **Computing in Science & Engineering**, v. 9, n. 3, p. 10-20, 2007.

PINTO, Mateus Ferreira. **Projeto de aplicativo para auxílio no transporte escolar**. 2017. 62 f. Trabalho de Conclusão de Curso (Tecnologia em Análise e Desenvolvimento de Sistemas) – Faculdade de Tecnologia de Americana, Americana, 2017.

PIOVESAN, Armando; TEMPORINI, Edméa Rita. Pesquisa exploratória: procedimento metodológico para o estudo de fatores humanos no campo da saúde pública. **Revista de Saúde Pública**, v. 29, n. 4, p. 318-325, 1995.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

SOUSA, Rafael Pereira de; BARBOSA, Ricardo Antunes; HINZ, Simone Pontes Stahl. Solução on-line para fiscalização de ônibus fretados. In: SIMPÓSIO DOS PROFISSIONAIS DA UNICAMP (SIMTEC), 8., 2022, Campinas. **Anais** [...]. Campinas: Unicamp, 2022.

TANDEL, Sayali Sunil; JAMADAR, Abhishek. Impact of Progressive Web Apps on Web App Development. **International Journal of Innovative Research in Science, Engineering and Technology**, v. 7, n. 9, p. 9439-9444, 2018.

VANESCOLA. **VanEscola**: sistema para gestão de vans escolares. [S. l.], 2026. Disponível em: https://www.vanescola.com/. Acesso em: 20 set. 2026.

VASCONCELLOS, Eduardo Alcântara de. A força política dos apoiadores do automóvel é muito superior à dos usuários do transporte coletivo. [Entrevista concedida a] IHU On-Line. **Instituto Humanitas Unisinos — IHU**, São Leopoldo, 15 set. 2014. Disponível em: https://www.ihu.unisinos.br/535225. Acesso em: 20 set. 2026.